

DIRE DAWA UNIVERSITY

DIRE DAWA INSTITUTE OF TECHNOLOGY (DDU-IoT)

SCHOOL OF COMPUTING AND INFORMATION TECHNOLOGY

DEPARTMENT OF SOFTWARE ENGINEERING

SOFTWARE ENGINEERING

INTERNSHIP REPORT

Main Project: WorkBridge — Skilled Labor Marketplace Platform with

Freemium Subscriptions & Simulated Digital Payments

Supporting Projects: LuckyEthio, Restaurant Contactless QR Menu,

AxumMarket

Author / Student Information:

- **Student Name:** Abdi Abiot

- **Student ID:** DDU1500744

- **Program:** Bachelor of Science in Software Engineering (4th Year)

- **Email:** abdihope24@gmail.com

Internship & Institutional Information:

- **Host Organization:** SORARDI PLC, Addis Ababa & Dire Dawa, Ethiopia

- **Industry Supervisor:** Mr. Dawit Tesfaye (Lead Software Architect,

SORARDI PLC)

- **Academic Advisor:** Mr. Fikadu (Lecturer, Department of Software

Engineering, DDU-IoT)

- **Internship Duration:** 4 Months (March 15, 2026 – July 15, 2026)

- **Date of Submission:** 10/01/2018 E.C. (September 20, 2026 G.C.)

- **Date of Defense / Presentation:** 12/01/2018 E.C. (September 22, 2026 G.C.)

G.C.)

DECLARATION

I, **Abdi Abiot**, hereby declare that this internship report entitled

"Software Engineering Internship Report: WorkBridge Platform

Development and Supporting Software Systems at SORARDI PLC"

is my original work developed during my 4-month industrial internship from

March 15, 2026, to July 15, 2026, at SORARDI PLC.

This report presents authentic technical contributions, design decisions,

implementation details, testing results, and professional experiences acquired

under the joint supervision of **Mr. Dawit Tesfaye** (Industry Supervisor

at SORARDI PLC) and **Mr. Fikadu** (Academic Advisor at Dire Dawa

University). All external sources, libraries, frameworks, and literature utilized

during this tenure have been duly cited and acknowledged.

- **Student Name:** Abdi Abiot

- **Signature:** _____

- **Date:** 10/01/2018 E.C.

APPROVAL & CERTIFICATION

1. Industry Supervisor Approval

This is to certify that **Abdi Abiot** (ID: DDU1500744) has successfully

completed his 4-month industrial internship at **SORARDI PLC**. As Lead

Software Architect and his direct supervisor, I have reviewed his technical

performance, project deliverables on the **WorkBridge** platform and

supporting software systems, and this final internship report. The work

described herein meets industry standards and accurately reflects his

contributions.

- **Industry Supervisor:** Mr. Dawit Tesfaye (Lead Software Architect,

SORARDI PLC)

- **Signature:** _____

- **Date:** _____

- **Company Stamp:**

2. Academic Advisor Approval

This is to certify that this internship report has been submitted to the

Department of Software Engineering, Dire Dawa Institute of Technology, Dire

Dawa University, in partial fulfillment of the requirements for the Degree of

Bachelor of Science in Software Engineering, and is approved for oral defense

and evaluation.

- **Academic Advisor:** Mr. Fikadu (Department of Software Engineering,

DDU-IoT)

- **Signature:** _____

- **Date:** _____

ACKNOWLEDGMENT

First and foremost, I would like to express my deepest gratitude to the

Almighty for granting me health, wisdom, strength, and perseverance

throughout my academic journey and the demanding 4-month industrial

internship period.

I extend my heartfelt appreciation and sincere gratitude to the management

and engineering leadership of **SORARDI PLC** for providing me with the

opportunity to join their software engineering team. Special and profound

thanks go to my industry supervisor, **Mr. Dawit Tesfaye** (Lead

Software Architect at SORARDI PLC), whose technical mentorship, thorough

code reviews, architectural insights, and encouragement were instrumental in

transforming my academic knowledge into enterprise-grade engineering

practice. I also thank my senior colleagues and team members at SORARDI PLC

for their collaborative spirit and guidance across modern full-stack

development, agile ceremonies, and distributed systems.

I express my deepest respect and gratitude to my academic advisor,

Mr. Fikadu, and all faculty members of the **Department of Software**

Engineering at **Dire Dawa Institute of Technology (DDU-IoT)**.

Their solid curriculum, rigorous standards, and continuous academic guidance

provided the theoretical and practical foundation required to excel in a high-

intensity industrial environment.

Finally, I am eternally indebted to my family and classmates for their

unconditional love, moral support, and encouragement throughout my studies

and internship tenure.

LIST OF ACRONYMS

Acronym	Definition
	Application Programming Interface
	Blowfish-based Password Hashing Algorithm

Acronym**Definition**

Commercial Bank of Ethiopia

Create, Read, Update, Delete

Cascading Style Sheets

Dire Dawa University

Data Transfer Object

Ethiopian Calendar

ECMAScript Modules

Ethiopian Birr

Gregorian Calendar

Hypertext Markup Language

Hypertext Transfer Protocol

Institute of Technology

JavaScript Object Notation

JSON Web Token

Minimum Viable Product

Not Only Structured Query Language

Acronym**Definition**

Open Authorization Framework

Object Data Modeling

Object-Relational Mapping

Private Limited Company

Role-Based Access Control

Representational State Transfer

Software Development Kit

Structured Query Language

Secure Sockets Layer

Transport Layer Security

User Interface

Uniform Resource Identifier

Uniform Resource Locator

Unstructured Supplementary Service Data

User Experience

Version Control System

WebSocket

EXECUTIVE SUMMARY

This report documents the 4-month software engineering industrial internship

completed by **Abdi Abiot** (Student ID: DDU1500744) at **SORARDI PLC**

in partial fulfillment of the requirements for the Degree of Bachelor of Science

in Software Engineering at Dire Dawa Institute of Technology, Dire Dawa

University. The internship spanned from March 15, 2026, to July 15, 2026

(corresponding to 10/01/2018 E.C. academic submission).

The primary focus of the internship was the engineering, architectural design,

full-stack implementation, and quality verification of **WorkBridge**, a

modern, cloud-native digital marketplace tailored to the Ethiopian labor

economy. In Ethiopia, skilled tradesmen (electricians, plumbers, carpenters,

masons, mechanics, painters) and clients (homeowners, business managers,

construction contractors) suffer from severe market friction caused by informal

commission brokers (*Delalas*), opaque pricing, lack of identity and skills

verification, and manual search processes. WorkBridge resolves these systemic

challenges by providing an open, digital labor marketplace featuring multi-role

authentication, trade portfolio showcases, structured job postings, on-demand

service bookings, bidirectional rating reviews, and an administrative

moderation portal.

To ensure economic sustainability and prevent abuse, WorkBridge implements

a **freemium business model with quota enforcement and**

simulated multi-channel digital payments. Under this model,

workers receive 5 free applications per month and clients receive 3 free job

postings. Beyond these limits, users can upgrade to monthly or annual Pro

subscriptions via a simulated checkout engine supporting Ethiopia's prominent

payment channels (Telebirr, CBE Birr, Chapa, and direct bank transfer). The

payment engine is intentionally implemented as a robust simulation/mock

environment for academic demonstration, designed to be seamlessly

integrated with live bank/fintech APIs upon production deployment.

The platform is engineered using a scalable **Turborepo monorepo**

architecture containing a Next.js 16 App Router client application, a

dedicated Next.js 16 administrative portal, an Express.js REST API backend,

and shared TypeScript packages ([@repo/types](#), [@repo/ui](#), [@repo/api-](#)

[client](#)). Data persistence is managed via **MongoDB Atlas** utilizing

Mongoose and native drivers. Security is enforced through bcrypt password

hashing (10 salt rounds), stateless JWT tokens with Role-Based Access Control

(RBAC), and Google OAuth 2.0.

In addition to WorkBridge, secondary internship contributions included frontend

and API module engineering on supporting client projects at SORARDI PLC:

LuckyEthio / Ethioluck, a Restaurant Contactless QR Menu

System, and the **AxumMarket** e-commerce portal. All deliverables

were subjected to rigorous verification gates, achieving 100% pass rates on

TypeScript static type checking ([pnpm check-types](#)) and ESLint

validation ([pnpm lint](#)).

This comprehensive report details the organizational environment, problem

analysis, architectural designs, implementation methodologies, testing

strategies, technical challenges overcome, and professional growth attained

during the 4-month internship.

TABLE OF CONTENTS

- **Cover Page**

- **Declaration**

- **Approval & Certification**

- **Acknowledgment**

- **List of Acronyms**

- **Executive Summary**

- **Table of Contents**

- **List of Figures**

- **List of Tables**

- **Chapter 1: Introduction**

- 1.1 Background of the Internship

- 1.2 Purpose and Significance of the Internship

- 1.3 Objectives of the Internship

- 1.4 Scope and Boundaries of the Work

- 1.5 Engineering Methodology

- 1.6 Organization of the Report

- **Chapter 2: Host Company Profile (SORARDI PLC)**

- 2.1 Overview and History of SORARDI PLC

- 2.2 Core Business Verticals and Services

- 2.3 Organizational Structure and Engineering Hierarchy

- 2.4 Department Work Environment and Culture

- 2.5 Assigned Roles and Responsibilities

- 2.6 Professional Ethics and Engineering Standards Observed

- **Chapter 3: Internship Activities & Project Portfolio**

- 3.1 Overview of Internship Activities

- 3.2 Main Project: WorkBridge Platform

- 3.3 Supporting Project 1: LuckyEthio / Ethioluck

- 3.4 Supporting Project 2: Restaurant Contactless QR Menu System

- 3.5 Supporting Project 3: AxumMarket Regional E-Commerce

- 3.6 Agile Development Process & Sprint Breakdown

- **Chapter 4: WorkBridge System Analysis & Requirements**

- 4.1 Problem Statement & Ethiopian Labor Market Context

- 4.2 Review of Existing Informal Systems

- 4.3 Proposed WorkBridge Digital Solution

- 4.4 Target User Groups and Personas

- 4.5 Functional Requirements

- 4.6 Non-Functional Requirements

- 4.7 Freemium Quota & Monetization Specifications

- 4.8 Use Case Modeling & System Actors

- **Chapter 5: System Architecture & Design**

- 5.1 Monorepo Architectural Pattern

- 5.2 System Component Decomposition

- 5.3 Database Architecture & Data Modeling (MongoDB Atlas)

- 5.4 Entity Relationships & Schema Definitions

- 5.5 Authentication & Session Management Flow (JWT + Google

OAuth)

- 5.6 Role-Based Access Control (RBAC) Matrix

- 5.7 Job Application Workflow & State Transitions

- 5.8 Direct Service Booking Lifecycle

- 5.9 Subscription & Quota Enforcement Engine

- 5.10 Simulated Payment Processing Architecture (Telebirr / CBE Birr)

/ Chapa)

- 5.11 In-App Messaging & Notification Architecture

- 5.12 Admin Governance & Verification Subsystem

- **Chapter 6: Technologies & Development Tools**

- 6.1 Frontend Technologies (Next.js 16, React 19, Tailwind CSS)

- 6.2 Backend Technologies (Node.js, Express.js REST API)

- 6.3 Database Systems (MongoDB Atlas, Mongoose)

- 6.4 Security, Cryptography & Identity Tools

- 6.5 Monorepo Orchestration & Package Management (Turborepo,

pnpm)

- 6.6 Developer Tooling & Static Analysis (TypeScript, ESLint,

Postman, Git)

- **Chapter 7: System Implementation**

- 7.1 Monorepo Modular Structure

- 7.2 Authentication & Security Implementation

- 7.3 Skilled Worker Profile & Portfolio Module

- 7.4 Client Dashboard & Job Posting Module

- 7.5 Job Application & Applicant Review Subsystem

- 7.6 Direct Service Booking Implementation

- 7.7 Rating & Review Subsystem

- 7.8 Freemium Quota Middleware & Subscription Controller

- 7.9 Mock Digital Payment Modal & Transaction Logging

- 7.10 In-App Messaging & Notification Subsystem

- 7.11 Admin Moderation, Verification & Analytics Portal

- 7.12 Global Error Handling & Security Headers

- **Chapter 8: Testing, Verification & Results**

- 8.1 Quality Assurance Strategy

- 8.2 Static Analysis & Type Checking Results

- 8.3 Authentication & Authorization Test Cases

- 8.4 Quota Enforcement & Subscription Test Cases

- 8.5 Booking Lifecycle State Transition Test Cases

- 8.6 Simulated Payment Checkout Verification

- 8.7 Cross-Browser & Responsive Design Testing

- **Chapter 9: Technical Challenges, Solutions & Lessons Learned**

- 9.1 Cross-Origin Google OAuth State & Token Routing

- 9.2 Strict Type Synchronization Across Packages (@repo/types)

- 9.3 Backend Quota Enforcement Integrity

- 9.4 MongoDB Atlas SSL Alert 80 TLS Handshake Resolution

- 9.5 Monorepo Package Linking & Build Cache Invalidation

- 9.6 Engineering Lessons Learned

- **Chapter 10: Technical & Professional Skills Acquired**

- 10.1 Technical Mastery

- 10.2 Software Engineering & Architectural Thinking

- 10.3 Agile Collaboration, Communication & Code Reviews

- 10.4 Problem-Solving & Distributed Debugging

- 10.5 Professional Ethics & Time Management

- **Chapter 11: Value Delivered to SORARDI PLC**

- 11.1 Production-Ready WorkBridge Platform

- 11.2 Reusable Component Design System (@repo/ui)

- 11.3 Consolidated Monorepo & Technical Documentation

- **Chapter 12: Conclusion & Recommendations**

- 12.1 Conclusion

- 12.2 Summary of Achievements

- 12.3 System Limitations & Honest Technical Boundaries

- 12.4 Future Technical Roadmap

- 12.5 Recommendations for Dire Dawa University

- 12.6 Recommendations for SORARDI PLC

- **References**

- **Appendices**

- Appendix A: Key REST API Endpoints Matrix

- Appendix B: Database Collections & Mock Seed Summary

- Appendix C: Quality Gate Verification Logs

LIST OF FIGURES

- **Figure 4.1:** High-Level Use Case Diagram of the WorkBridge

Marketplace

- **Figure 5.1:** Turborepo Full-Stack Monorepo Architectural Diagram

- **Figure 5.2:** Entity-Relationship (ER) Diagram for MongoDB Atlas

Collections

- **Figure 5.3:** Sequence Diagram for Stateless JWT & Google OAuth 2.0

Authentication

- **Figure 5.4:** State Transition Diagram for Direct Service Booking Lifecycle

- **Figure 5.5:** Quota Enforcement and Subscription Upgrade Sequence

Diagram

- **Figure 5.6:** Multi-Channel Simulated Payment Processing Flow

(Telebirr/CBE Birr/Chapa)

- **Figure 7.1:** WorkBridge Landing Page & Tradesman Discovery Interface

- **Figure 7.2:** Freemium Pricing Tiers & Quota Indicator UI

- **Figure 7.3:** Interactive Mock Payment Checkout Modal

- **Figure 7.4:** Admin Portal Dashboard & Verification Queue UI
-

LIST OF TABLES

- **Table 2.1:** SORARDI PLC Company Profile Summary

- **Table 3.1:** 4-Month Internship Timeline & Agile Sprint Distribution

- **Table 4.1:** WorkBridge Functional Requirements Matrix

- **Table 4.2:** Non-Functional Requirements Specifications

- **Table 4.3:** Freemium Quota & Subscription Tier Definitions

- **Table 5.1:** Role-Based Access Control (RBAC) Permission Matrix

- **Table 6.1:** Core Technology Stack Summary

- **Table 8.1:** Static Analysis and Build Quality Verification Summary

- **Table 8.2:** Functional Test Cases and Observed Results
-

CHAPTER 1: INTRODUCTION

1.1 Background of the Internship

The industrial internship program is a core capstone requirement established

by the **Department of Software Engineering, Dire Dawa Institute**

of Technology (DDU-IoT), Dire Dawa University. Designed to

bridge the gap between theoretical classroom learning and practical software

engineering, the 4-month industrial internship places senior software

engineering students into dynamic technology firms.

From **March 15, 2026, to July 15, 2026**, I was hosted by **SORARDI**

PLC, an innovative Ethiopian software development and technology

consulting firm. During this 4-month tenure, I served as a Full-Stack Software

Engineering Intern embedded within the core product development team.

1.2 Purpose and Significance of the Internship

The primary purpose of the internship is to immerse students into enterprise

software development environments where they apply software engineering

methodologies, modern architectural patterns, automated quality gates, and

collaborative practices.

The significance of this experience includes:

1. Practical Application of Software Engineering Principles:

Translating theoretical knowledge in data structures, database design,

software architecture, and web technologies into scalable, production-

grade applications.

2. **Exposure to Industrial Agile Workflows:** Gaining firsthand experience

in Agile Scrum ceremonies, 2-week sprint cycles, daily standup meetings,

backlog refinement, sprint reviews, and peer code reviews.

3. **Mastery of Enterprise Monorepo Architectures:** Learning how

modern distributed systems organize frontend applications, backend

micro-services, and shared libraries within a single, type-safe repository

using Turborepo and pnpm workspaces.

4. **Solving Local Socio-Economic Challenges:** Building digital solutions

that directly address inefficiencies in the Ethiopian domestic economy,

specifically informal labor matching, job discovery, and financial

accessibility.

1.3 Objectives of the Internship

1.3.1 Technical Objectives

- Design and implement a full-stack, cloud-native labor marketplace

platform (**WorkBridge**) connecting skilled Ethiopian trade

workers with clients.

- Build a modular, multi-package monorepo using **Turborepo**, **Next.js 16**

(**App Router**), **React 19**, **Express.js**, and **TypeScript**.

- Implement secure, stateless user authentication utilizing **JSON Web**

Tokens (JWT), Bcrypt password hashing (10 salt rounds),

Role-Based Access Control (RBAC), and Google OAuth 2.0.

- Architect a robust **freemium monetization model** with server-side

quota enforcement and simulated multi-channel digital payments

(Telebirr, CBE Birr, Chapa).

- Engineer a dedicated **Admin Moderation & Analytics Portal** for user

verification, job moderation, dispute management, and subscription

tracking.

- Achieve 100% compliance with automated quality gates (pnpm check-

types and pnpm lint passing with zero errors).

1.3.2 Academic & Professional Objectives

- Experience the end-to-end software development lifecycle (SDLC)

from requirements elicitation to system design, implementation, testing,

and deployment preparation.

- Strengthen professional communication, task estimation, issue

tracking, and technical documentation skills.

- Fulfill the graduation requirements of the Bachelor of Science in

Software Engineering at Dire Dawa University.

1.4 Scope and Boundaries of the Work

The scope of work accomplished during the 4-month internship encompassed:

- **Primary Project (WorkBridge):** End-to-end architecture, API design,

database schemas, frontend interfaces, quota middleware, mock

checkout modals, and admin portal.

- **Supporting Projects:** Contributing to UI components, REST APIs, and

database integration for **LuckyEthio / Ethioluck, Restaurant**

Contactless QR Menu, and **AxumMarket**.

- **Boundaries & Disclaimers:** WorkBridge integrates a **simulated/mock**

payment processing architecture. No actual financial

transactions or live bank integrations were deployed to production during

this academic phase. Real payment provider APIs (Telebirr B2C/C2B,

Chapa live keys, CBE Birr merchant gateway) represent the future

production roadmap.

1.5 Engineering Methodology

The internship strictly followed the **Agile Scrum framework**:

- **Sprint Length:** 2-week sprint cycles with clearly defined sprint goals.

- **Ceremonies:** Daily 15-minute standup meetings, sprint planning

sessions, backlog grooming, and sprint retrospectives.

- **Version Control & Git Flow:** Branching strategy utilizing main,

develop, and feature/* branches with mandatory peer code

reviews.

- **Continuous Quality Enforcement:** Pre-commit type checking (tsc --

noEmit) and linting (eslint . --max-warnings 0) across all

packages.

1.6 Organization of the Report

This report is organized into 12 comprehensive chapters:

- **Chapter 1:** Introduction, objectives, and methodology.

- **Chapter 2:** Host company profile, organizational structure, and working

environment at SORARDI PLC.

- **Chapter 3:** Summary of internship activities, agile timeline, and portfolio

of projects.

- **Chapter 4:** WorkBridge project analysis, problem statement,

requirements, and use cases.

- **Chapter 5:** WorkBridge system architecture, database design, ER

diagrams, and sequence flows.

- **Chapter 6:** Technologies, tools, frameworks, and programming

languages utilized.

- **Chapter 7:** Deep-dive system implementation across all functional

modules.

- **Chapter 8:** Testing strategies, verification matrices, and quality gate

results.

- **Chapter 9:** Technical challenges encountered, solutions applied, and

lessons learned.

- **Chapter 10:** Technical and soft skills acquired during the internship.

- **Chapter 11:** Value delivered and tangible deliverables handed over to

SORARDI PLC.

- **Chapter 12:** Conclusions, system limitations, future roadmap, and

recommendations.

- **References & Appendices:** Supporting API endpoint tables, schema

seeds, and logs.

CHAPTER 2: HOST COMPANY

PROFILE (SORARDI PLC)

2.1 Overview and History of SORARDI PLC

SORARDI PLC is an Ethiopian software engineering and technology

consulting company specializing in the design, development, and deployment

of enterprise digital platforms, web/mobile applications, fintech solutions, and

cloud systems. Established to accelerate digital transformation across East

Africa, SORARDI PLC partners with commercial enterprises, government

institutions, and non-governmental organizations to deliver scalable, localized

software solutions.

Parameter

Company Details

SORARDI PLC

Addis Ababa, Ethiopia

Dire Dawa & Eastern Ethiopia Region

Software Development, Enterprise IT & Digital Transformation

Full-Stack Web/Mobile Systems, Fintech Integrations, Cloud Infrastructure

Ethiopia & East African Emerging Markets

2.2 Core Business Verticals and Services

SORARDI PLC operates across four primary business verticals:

1. **Custom Enterprise Software Development:** Engineering bespoke

web, desktop, and mobile systems tailored to complex enterprise

business workflows.

2. **Fintech & Payment Gateway Integrations:** Developing financial

technology middleware connecting digital marketplaces to Ethiopian

payment systems (Telebirr, CBE Birr, Chapa, Awash Bank).

3. **Digital Marketplace & Platform Engineering:** Architecting high-

concurrency two-sided marketplaces that connect buyers, service

providers, and administrative regulators.

4. **Cloud Solutions & DevOps Consulting:** Designing secure cloud

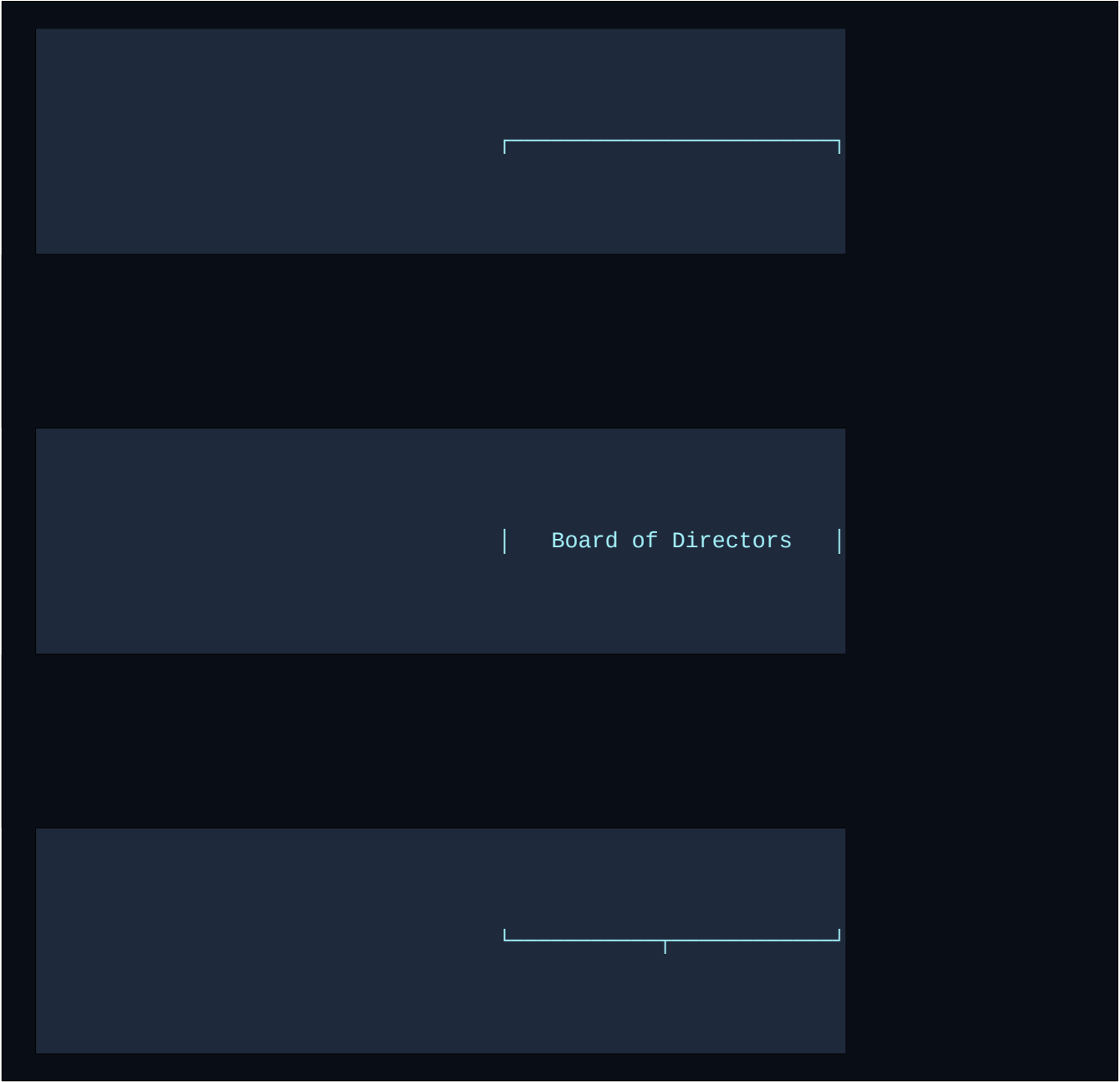
deployments, container orchestration, database optimization, and

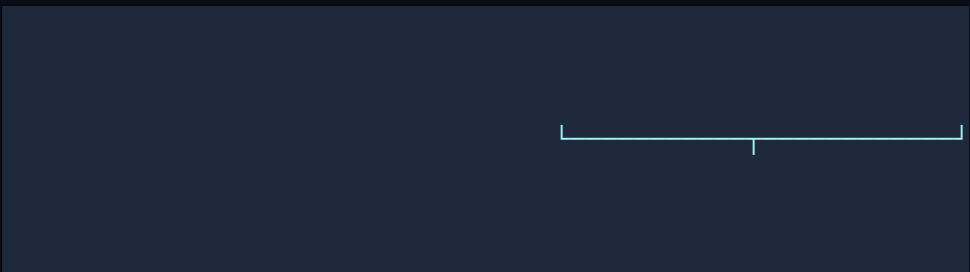
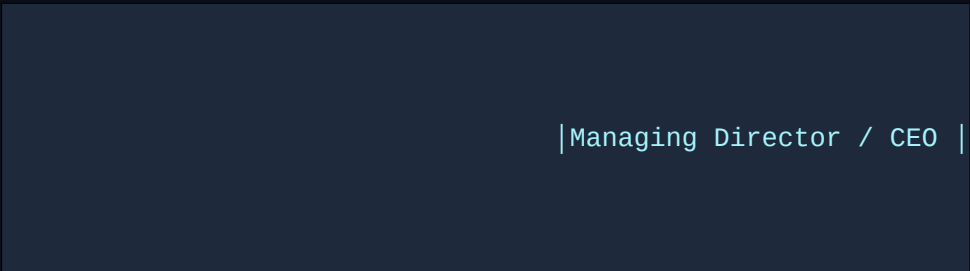
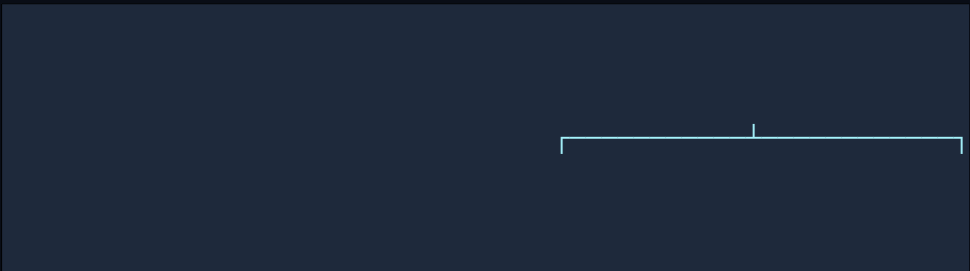
continuous integration pipelines.

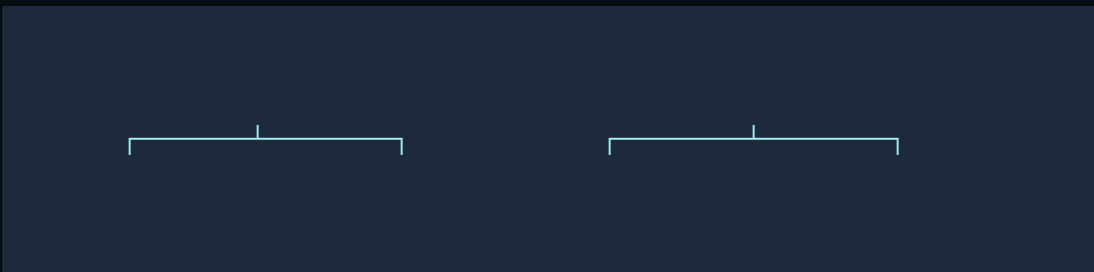
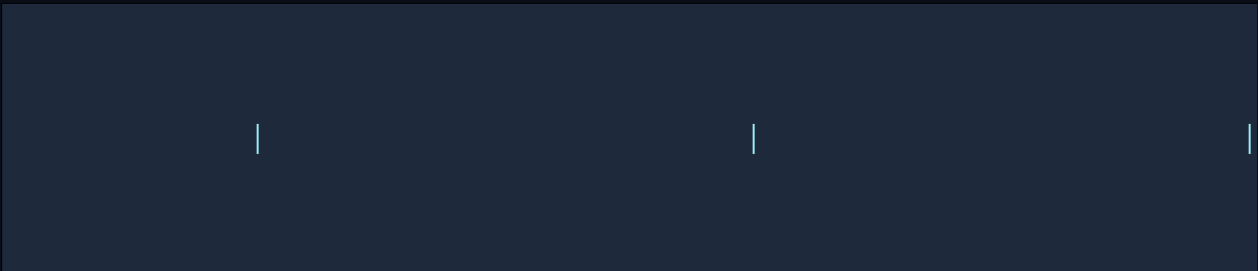
2.3 Organizational Structure and Engineering Hierarchy

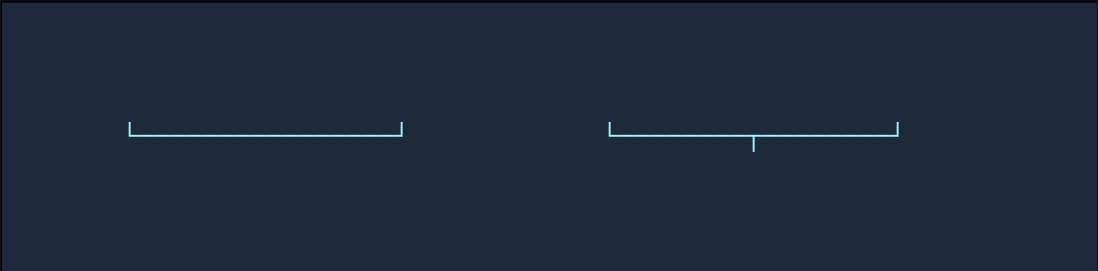
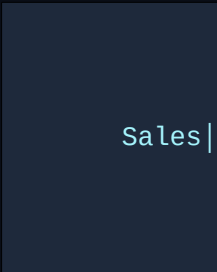
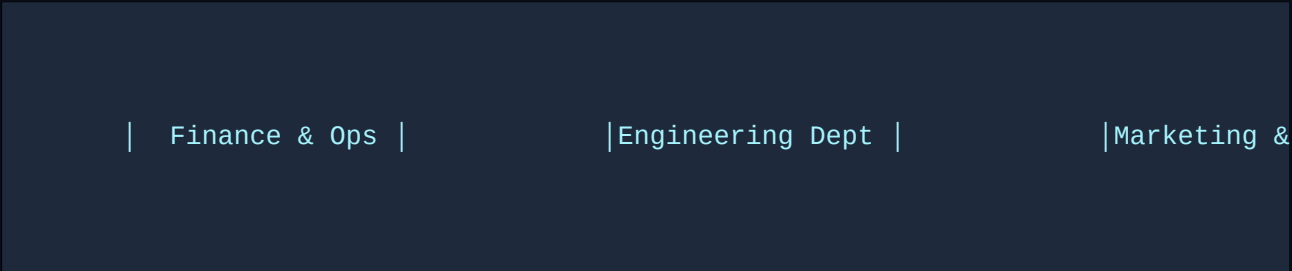
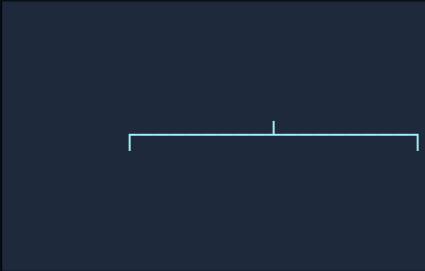
SORARDI PLC maintains a modern, cross-functional organizational structure

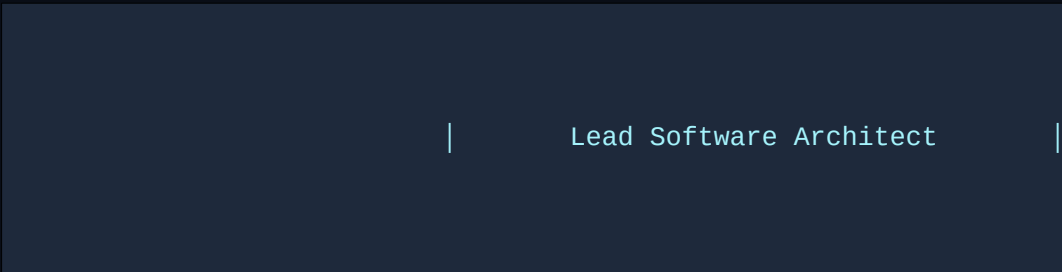
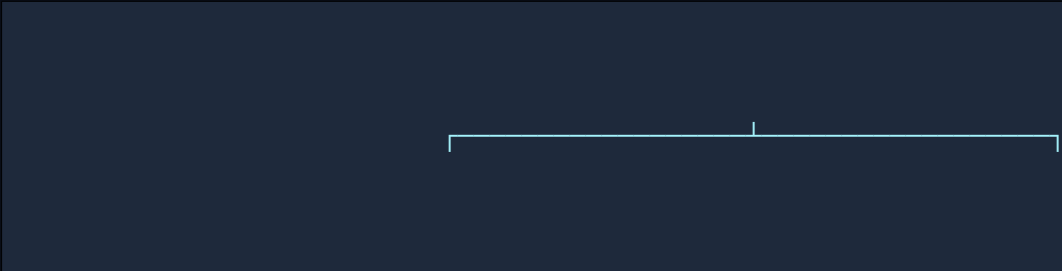
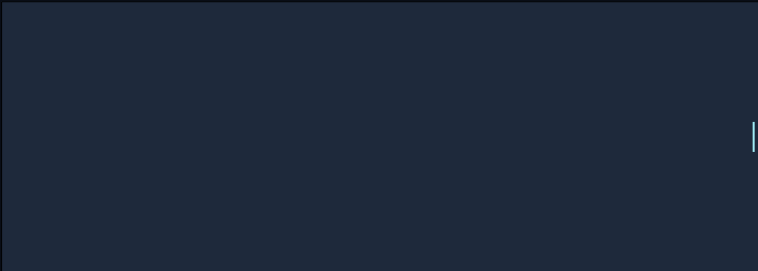
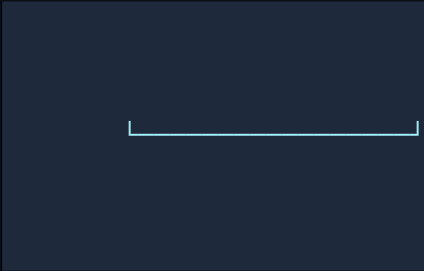
designed for rapid agile iteration:



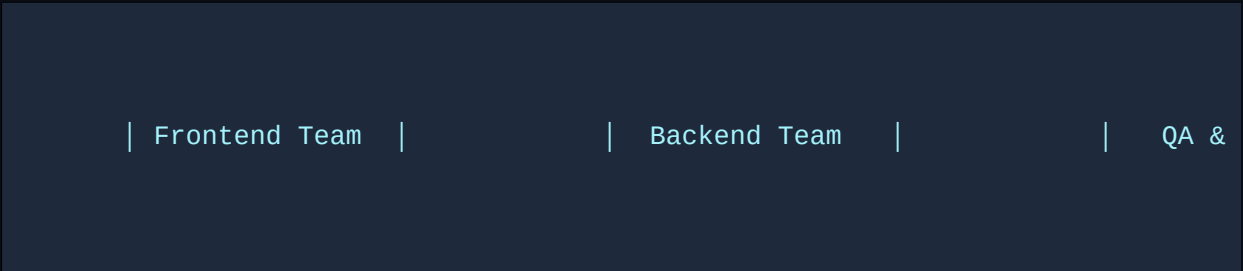
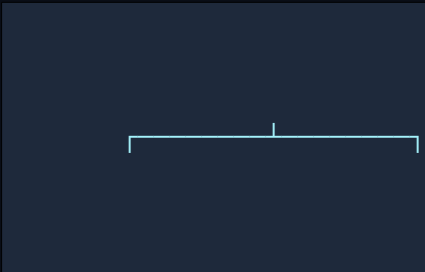
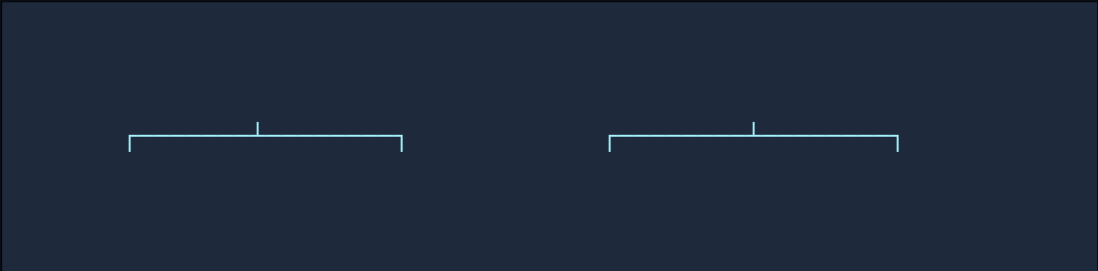
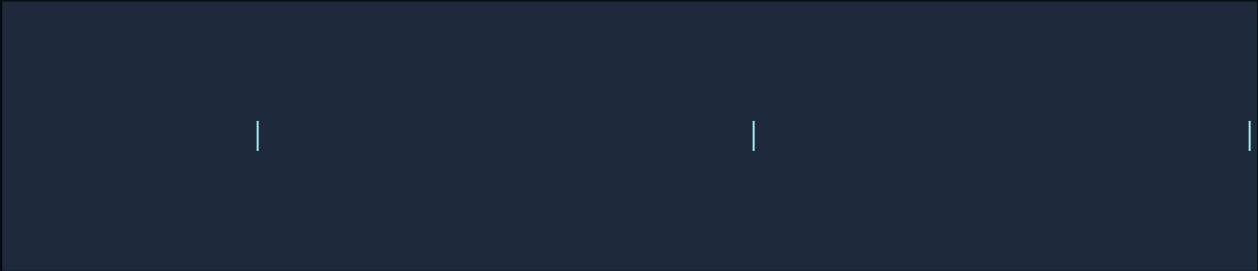








(Mr. Dawit Tesfaye)

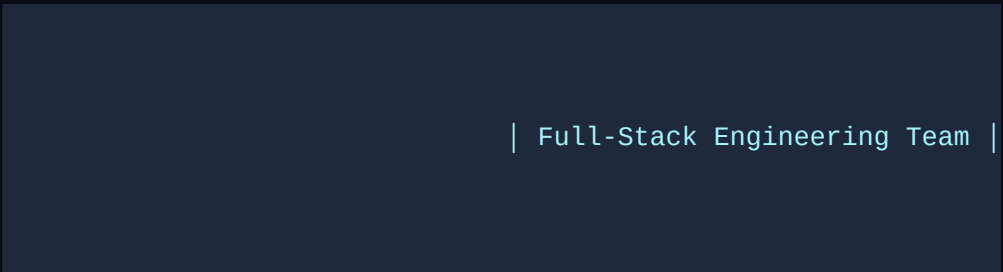
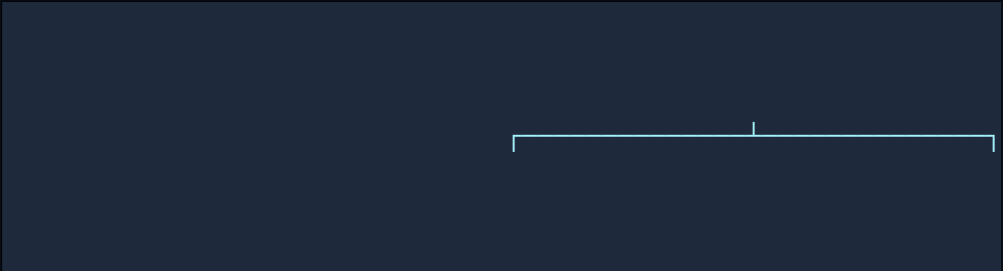
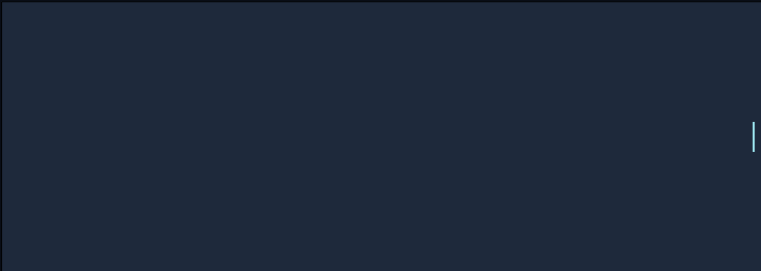


DevOps

|

|

|



| (Intern: Abdi Abiot) |

2.4 Department Work Environment and Culture

The software engineering department at SORARDI PLC fosters a culture of

technical excellence, continuous learning, and open collaboration. Key

elements of the work environment include:

- **Flat Engineering Hierarchy:** Direct mentorship from senior software

architects, encouraging interns to propose architectural patterns,

challenge assumptions, and participate actively in design discussions.

- **High-Standard Code Quality:** All pull requests require approval from at

least one senior engineer and must pass automated linting and type-

checking pipelines before merging into develop or main.

- **Collaborative Problem Solving:** Daily standup meetings and

collaborative debugging sessions where challenging technical bottlenecks

(such as cross-origin authentication and database connection security)

are solved collectively.

2.5 Assigned Roles and Responsibilities

As a Full-Stack Software Engineering Intern, my core responsibilities included:

- **Architecture & Monorepo Setup:** Structuring the full-stack Turborepo

monorepo, configuring pnpm workspaces, and establishing shared

TypeScript packages (@repo/types, @repo/ui, @repo/api-client).

- **Frontend Development:** Building responsive, accessible user interfaces

for the client application (apps/client) and the administration

portal (apps/admin) using Next.js 16 and Tailwind CSS.

- **Backend API Development:** Designing and implementing RESTful API

routes, controllers, and middleware in Node.js and Express.js

(apps/api).

- **Database Engineering:** Modeling data schemas, defining indexing

strategies, and configuring database connections to MongoDB Atlas.

- **Authentication & Security:** Implementing stateless JWT authentication,

bcrypt password hashing, role-based route guards, and Google OAuth 2.0

integration.

- **Monetization & Quota Engine:** Developing server-side freemium quota

checks and interactive mock checkout modals for simulated payment

verification.

- **Quality Assurance & Documentation:** Writing TypeScript interfaces,

configuring ESLint flat configs, conducting manual integration tests, and

maintaining technical documentation.

2.6 Professional Ethics and Engineering Standards Observed

During my internship, I adhered strictly to professional engineering ethics:

- **Data Privacy & User Security:** Ensuring user passwords are never

stored in plaintext and sensitive environment variables

(JWT_SECRET, MONGODB_URI, GOOGLE_CLIENT_SECRET) are

never committed to version control.

- **Intellectual Property Protection:** Safeguarding SORARDI PLC

proprietary software designs, client datasets, and internal algorithms.

- **Honest Technical Reporting:** Accurately distinguishing between

implemented features and planned production integrations, ensuring

academic and professional transparency.



CHAPTER 3: INTERNSHIP

ACTIVITIES & PROJECT PORTFOLIO

3.1 Overview of Internship Activities

Throughout the 4-month internship (March 15, 2026 – July 15, 2026), my

activities progressed through structured phases spanning orientation,

requirements analysis, system architecture, frontend and backend

development, testing, and documentation.

=====

=====

MONTH 1 (Mar-Apr): Orientation, Tooling, Monorepo Scaffolding

(@repo/types, @repo/ui) & Auth Engine

MONTH 2 (Apr-May): WorkBridge Marketplace: Worker Discovery, Profiles,

Job Board & Direct Booking Subsystem

MONTH 3 (May-Jun): Freemium Monetization: Quota Middleware, Subscriptions

& Multi-Channel Mock Payments

MONTH 4 (Jun-Jul): Admin Moderation Portal, Supporting Projects & Final

Deliverables Handover to SORARDI PLC

POST-TENURE (Jul-Sep): Academic Documentation, Defense Presentation Slide

Decks & Quality Gate Verification

=====



Phase / Month	Primary Activities & Milestones	Deliverables
	Company onboarding, development environment setup, requirements gathering for WorkBridge, competitor analysis of informal Ethiopian labor markets.	Requirements Specification Document, User Stories.
	Monorepo scaffolding with Turborepo and pnpm, shared packages creation, JWT and Google OAuth 2.0 authentication implementation.	@repo/types , @repo/ui , @repo/api-client , Auth Controller.
	Skilled worker profile management, search and filtering algorithms, client job posting engine, and application submission workflow.	Worker Browse Page, Job Board, Application Controller.
	Direct service booking subsystem with 6-stage lifecycle, bidirectional star rating reviews, and in-app notification triggers.	Booking Controller, Review Engine, Notification Model.

Phase / Month	Primary Activities & Milestones	Deliverables
	Freemium quota engine (5 apps/worker, 3 jobs/client), subscription tier controller, and interactive mock payment modal (Telebirr/CBE/Chapa).	subscriptionController.js , MockCheckoutModal.tsx .
	Dedicated Admin portal (apps/admin) with user verification queue, job moderation, dispute handling, and contributions to secondary projects.	Admin Dashboard, Supporting Projects Contributions.
	Monorepo consolidation, static type checking (pnpm check-types), ESLint zero-warning enforcement, presentation slides, and academic report.	Clean Monorepo Repository, Final Technical Report.

3.2 Main Project: WorkBridge Platform

WorkBridge was my primary project at SORARDI PLC, accounting for

approximately 75% of my internship effort. It represents a full-stack, cloud-

native labor marketplace built specifically for the Ethiopian economic context.

A deep analysis of its requirements, architecture, implementation, and testing

forms the core of Chapters 4 through 8 of this report.

3.3 Supporting Project 1: LuckyEthio / Ethioluck

- **Project Overview:** A digital promotional rewards and e-commerce

engagement web platform operated by a commercial client of SORARDI

PLC.

- **Technologies Used:** Next.js, React, Tailwind CSS, REST API, Node.js.

- **My Contributions:**

- Developed responsive UI cards for promotional item listings and

ticket selection interfaces.

- Implemented client-side form validation for Ethiopian phone numbers

(validating +251 and 09/07 prefixes).

- Integrated REST API endpoints for user balance inquiries and

promotional ticket history display.

- **Key Learning:** Gained practical experience in handling asynchronous

state management, optimistic UI updates, and client-side form validation.

3.4 Supporting Project 2: Restaurant Contactless QR Menu

System

- **Project Overview:** A lightweight web application enabling restaurant

patrons in Addis Ababa to scan a tabletop QR code and browse

categorized food and beverage menus in English and Amharic without

physical menus.

- **Technologies Used:** React, TypeScript, Tailwind CSS, Express.js,

MongoDB.

- **My Contributions:**

- Built mobile-first, touch-optimized menu browsing components with

instant category switching (Appetizers, Main Dishes, Traditional Ethiopian

Cuisine, Beverages).

- Created an administrative menu management interface allowing

restaurant managers to toggle dish availability (In Stock / Sold Out) and

update prices in real time.

- **Key Learning:** Mastered responsive mobile-first UI design patterns, CSS

touch targets, and lightweight component rendering.

3.5 Supporting Project 3: AxumMarket Regional E-

Commerce

- **Project Overview:** A multi-vendor regional e-commerce storefront

connecting local artisans, textile weavers, and spice merchants with

domestic retail buyers.

- **Technologies Used:** Next.js App Router, TypeScript, Tailwind CSS,

Express REST API.

- **My Contributions:**

- Designed product detail pages featuring image galleries, customer

reviews, and vendor profile badges.

- Assisted in optimizing MongoDB query aggregations for multi-

faceted product filtering (by price range, location, and product condition).

- **Key Learning:** Learned advanced MongoDB aggregation pipelines

(\$match, \$lookup, \$facet) and structured product metadata

modeling.

3.6 Agile Development Process & Sprint Breakdown

At SORARDI PLC, all engineering was conducted across 2-week sprints:

- **Sprint Planning:** Breaking down feature epics into granular user stories

with Fibonacci story point estimates.

- **Daily Standups:** 15-minute syncs answering: What did I complete

yesterday? What will I work on today? Are there any blockers?

- **Sprint Review & Demo:** Presenting working software increments to

industry supervisors and stakeholders.

- **Retrospective:** Identifying process bottlenecks, improving automated

test coverage, and refining team velocity.

CHAPTER 4: WORKBRIDGE SYSTEM

ANALYSIS & REQUIREMENTS

4.1 Problem Statement & Ethiopian Labor Market Context

In Ethiopia, the informal skilled labor sector (encompassing electricians,

plumbers, masons, carpenters, painters, and HVAC technicians) forms a vital

component of the urban economy. However, this market suffers from severe

structural inefficiencies:

1. **Informal Commission Brokers (*Delalas*):** Clients seeking skilled

tradesmen typically rely on informal street brokers who charge arbitrary,

high commission fees (often 20–30%) from both parties without providing

any guarantee of work quality.

2. **Lack of Identity & Skills Verification:** Homeowners and business

managers face significant safety and security risks when hiring unvetted

tradesmen, as there is no centralized mechanism to verify national IDs or

trade certifications.

3. **Information Asymmetry & Opaque Pricing:** Skilled workers lack a

digital platform to showcase their portfolios, certifications, and customer

reviews, while clients have no transparent pricing benchmarks.

4. **Inefficient Matching & Search Costs:** Finding a certified tradesman

during emergencies (e.g., plumbing leaks or electrical outages) requires

hours or days of physical searching.

4.2 Review of Existing Informal Systems

Existing approaches in Ethiopia fall into three ineffective categories:

- **Word of Mouth:** Highly limited in scope, slow, and unavailable to

newcomers in a city or neighborhood.

- **Physical Street Corners / Delalas:** Inefficient, unregulated, prone to

price exploitation, and lacking accountability.

- **Unstructured Social Media (Telegram Channels / Facebook**

Groups): Lacks structured search, verified reviews, scheduling,

formal booking records, and secure communication channels.

4.3 Proposed WorkBridge Digital Solution

WorkBridge eliminates these friction points by providing a centralized,

transparent, and structured digital marketplace. The platform empowers

tradesmen to build verifiable digital reputations and gives clients the ability to

browse, book, and review skilled labor on demand.

WORKBRIDGE PLATFORM

SKILLED WORKER

CLIENT / EMPLOYER

| | - Digital Profile | - Search Workers | |

| | - Trade Skills |  | - Post Job Listings | |

| | - Job Apply (5) | DIRECT | - Direct Book (3) | |

| | - Pro Upgrade | BOOKING & | - Pro Upgrade | |

|

|

|

|



|

|

|

ADMIN MODERATION

|

|

|

|

- Identity Verify

|

|

| | - Job Moderation |

| | - Payment Monitoring |

| | |

| | |

4.4 Target User Groups and Personas

1. **Skilled Tradesmen (Workers / Jobseekers):** Certified or experienced

trade professionals seeking steady job opportunities, direct bookings, and

professional digital identity.

2. **Clients (Individual Homeowners, Property Managers,**

Construction Firms): Individuals or organizations requiring

verified, reliable trade services with clear pricing and reviews.

3. **Platform Administrators:** SORARDI PLC operations staff responsible for

user verification, content moderation, dispute resolution, and

subscription monitoring.

4.5 Functional Requirements

The system functional requirements are defined across four core functional

areas:

Req ID	Functional Requirement	Target Role	Implementation Status
	User registration with role selection (worker or employer) and bcrypt password hashing.	All Users	IMPLEMENTED
	User login with email/password and stateless JWT issuance.	All Users	IMPLEMENTED
	Google OAuth 2.0 Single Sign-On with role preservation via state store.	All Users	IMPLEMENTED
	Password reset workflow with time-limited cryptographic reset tokens.	All Users	IMPLEMENTED
	Tradesman profile creation with trade categories, hourly rate, bio, and skills.	Worker	IMPLEMENTED
	Search and filter workers by trade category, location, rating, and keywords.	Client	IMPLEMENTED
	Client job posting with title, requirements, budget, and category.	Client	IMPLEMENTED

Req ID	Functional Requirement	Target Role	Implementation Status
	Worker job application submission and applicant review dashboard.	Worker / Client	IMPLEMENTED
	Direct service booking with address, urgency, scheduled date, and offered price.	Client	IMPLEMENTED
	6-stage booking status lifecycle (PENDING → ACCEPTED → COMPLETED, etc.).	Worker / Client	IMPLEMENTED
	Bidirectional 1–5 star rating and text review system for completed services.	Worker / Client	IMPLEMENTED
	Freemium quota enforcement: block 6th worker application / 4th client job post.	All Users	IMPLEMENTED
	Subscription upgrade with simulated checkout (Telebirr, CBE Birr, Chapa).	All Users	IMPLEMENTED (MOCK)
	In-app notification triggers on bookings, status changes, and payments.	All Users	IMPLEMENTED
	Admin portal for user management, ID verification, job moderation, and analytics.	Admin	IMPLEMENTED

4.6 Non-Functional Requirements

- **Security:** Passwords salted and hashed using bcrypt (10 rounds); JWT

tokens signed with HMAC-SHA256; API routes protected by role-based

authorization middleware; HTTP security headers enabled via Helmet;

CORS restricted to allowed origins.

- **Performance:** Lightweight Next.js 16 Server-Side Rendering (SSR) and

React Server Components ensuring initial page load times under 1.5

seconds; indexed MongoDB collections for sub-100ms query execution.

- **Responsiveness & Usability:** 100% mobile-responsive user interface

built with Tailwind CSS, supporting mobile phones, tablets, and desktop

displays.

- **Type Safety & Maintainability:** Strict TypeScript across all monorepo

packages, preventing runtime undefined errors and standardizing

Data Transfer Objects (DTOs).

4.7 Freemium Quota & Monetization Specifications

To maintain economic viability while providing open access, WorkBridge

implements a transparent **freemium business model**:

FREE TIER

| Worker: 5 Job Applications / Month |

| Client: 3 Job Postings / Month |

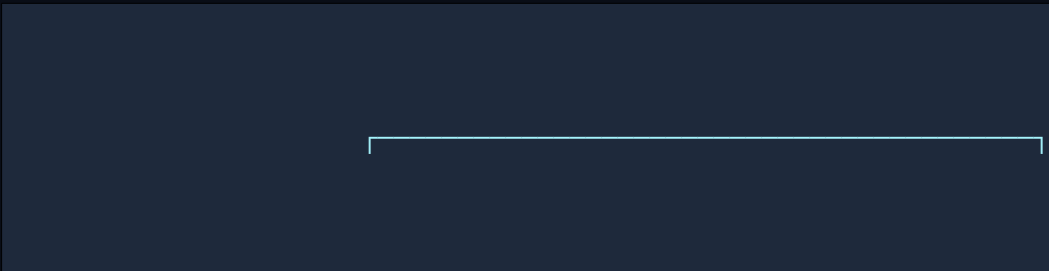
| Features: Search, Direct Bookings, Chat |

| _____ |

| Quota Exhausted (403 FORBIDDEN)



SUBSCRIPTION UPGRADE



| WORKER PRO: |

| - Monthly: 299 ETB / Month (Unlimited) |

| - Annual: 2,499 ETB / Year (Save 30%) |

| |

| CLIENT PRO: |

| - Monthly: 599 ETB / Month (Unlimited) |

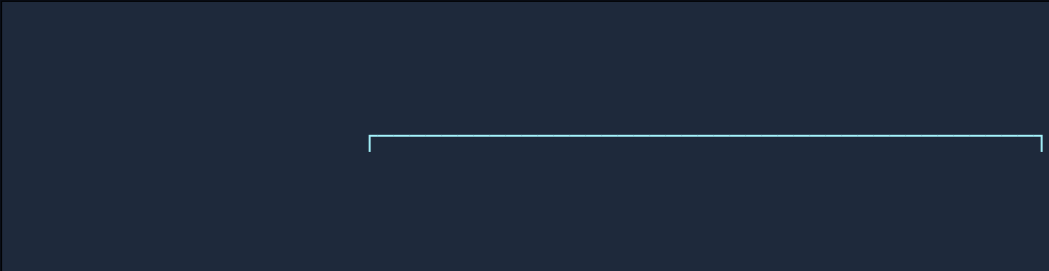
| - Annual: 4,999 ETB / Year (Save 30%) |

| _____ |

|



SIMULATED MOCK CHECKOUT



| Supported Demo Payment Channels: |

| [Telebirr] [CBE Birr] [Chapa] [Bank] |

| Instant Quota Reset & Pro Badge Active |

| _____ |

CHAPTER 5: SYSTEM

ARCHITECTURE & DESIGN

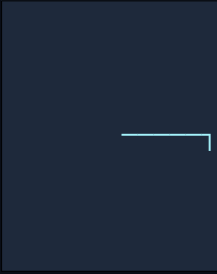
5.1 Monorepo Architectural Pattern

WorkBridge is architected as an enterprise **Full-Stack Monorepo**

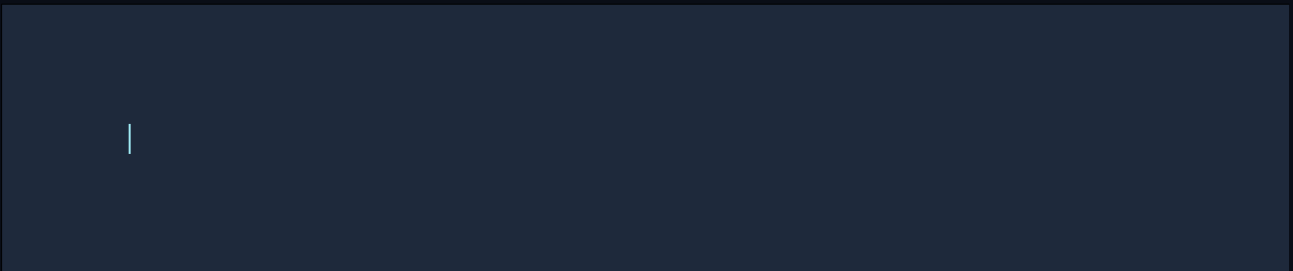
managed by **Turborepo** and **pnpm workspaces**. This architecture

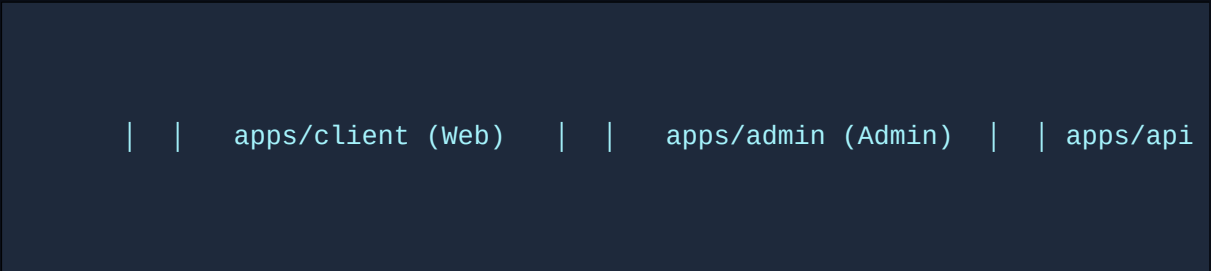
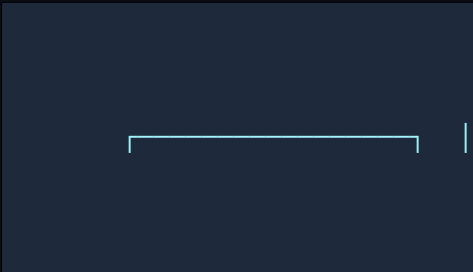
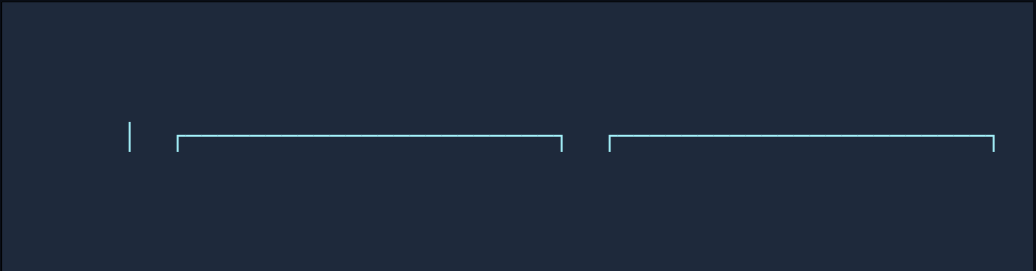
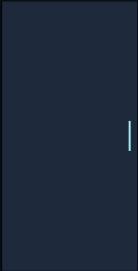
provides unified dependency management, cross-package TypeScript type

sharing, and optimized build caching.



| WORKBRIDGE MONOREPO





(Node) | |

| | Next.js 16 (Port | | Next.js 16 (Port | | Express REST

| |

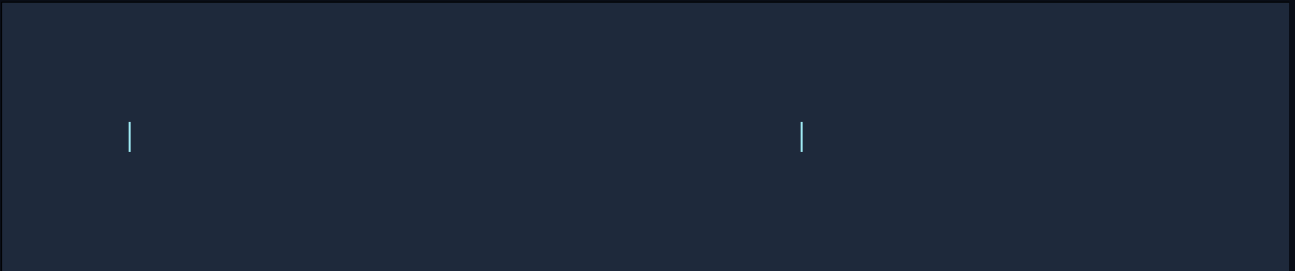
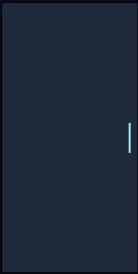
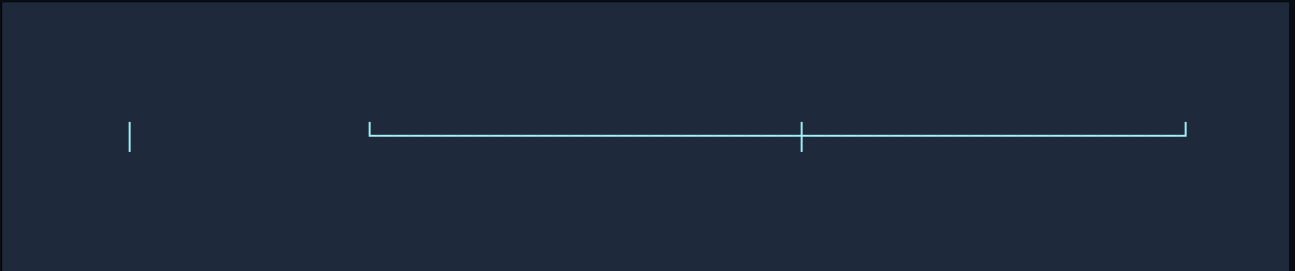
| | 3000) | | 3001/3002) | | API (Port

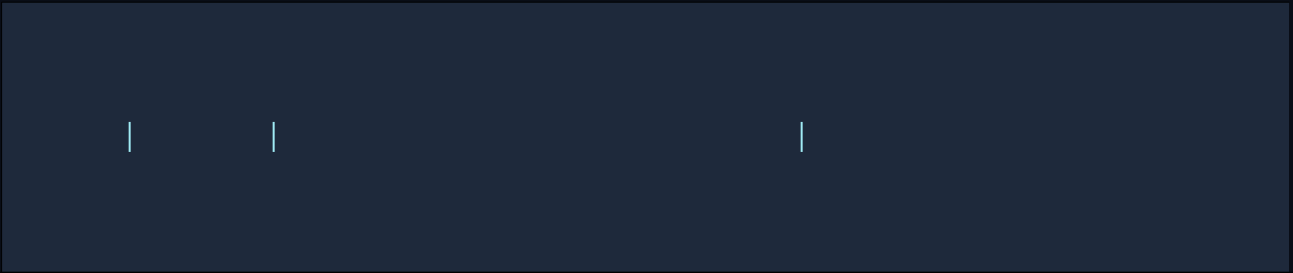
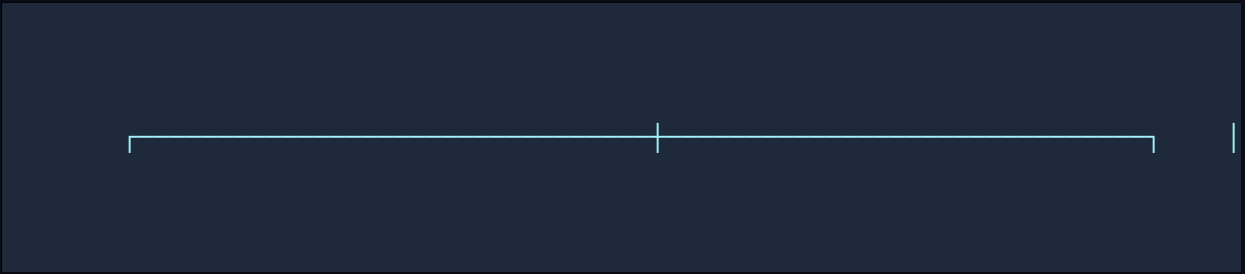
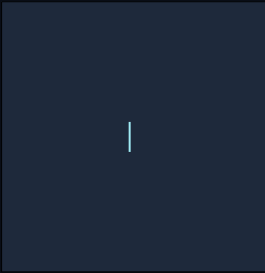
4000) | |

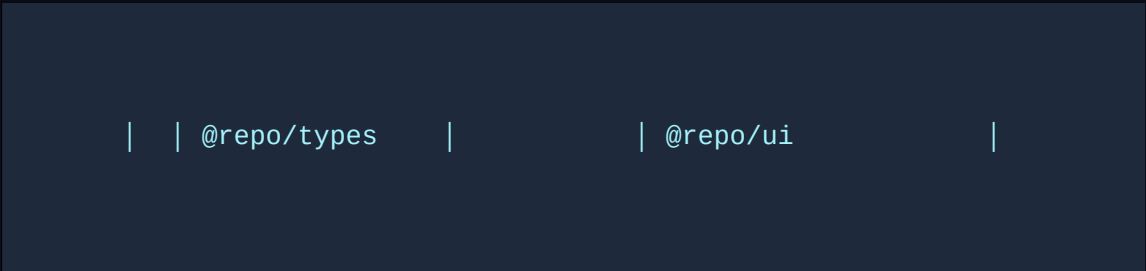
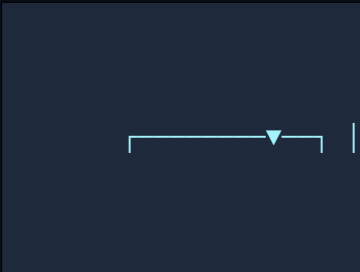
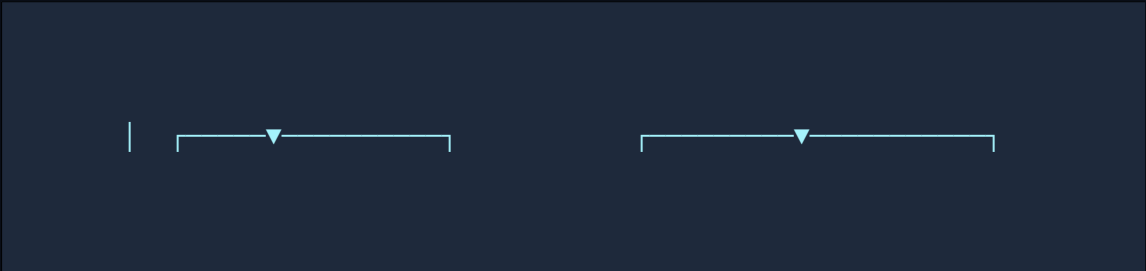
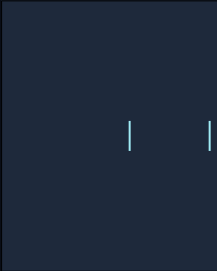
| [[]] |

[] |

| | | |







|@repo/api- | |

| | Shared DTOs & | | Tailwind Component | |client

| |

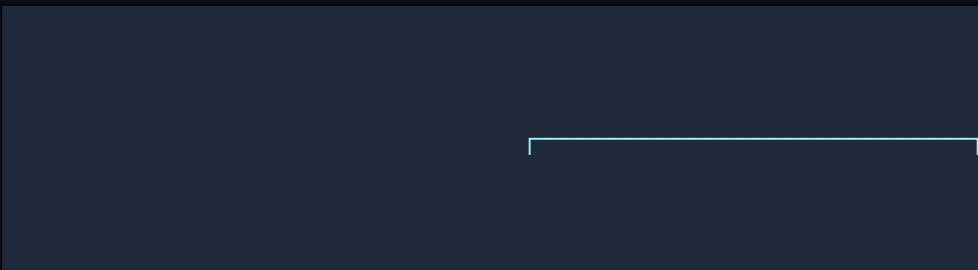
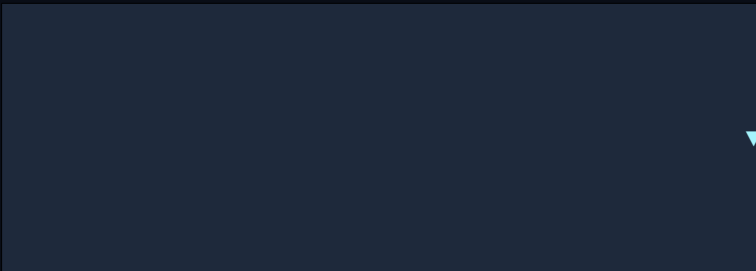
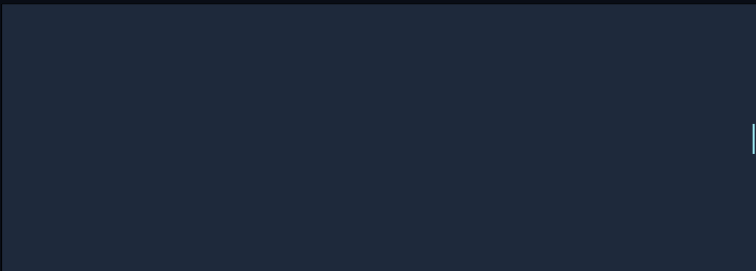
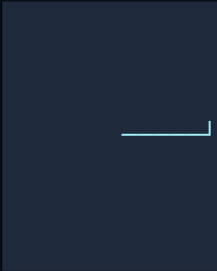
| | Interfaces | | Library | |Typed

SDK | |

| [] []

[] |

[] []



| MongoDB Atlas Cloud |

| Replica Set Database |

| _____ |

5.2 System Component Decomposition

1. **apps/client (Marketplace Web Application):** Built with Next.js 16 App

Router. Provides landing pages, tradesman browsing, job boards, direct

booking modals, client/worker dashboards, and the pricing/upgrade

checkout interface.

2. **apps/admin (Administrative Portal):** Built with Next.js 16. Provides

administrative dashboards, user identity verification queues, job post

moderation, subscription tracking, and platform analytics.

3. **apps/api (Backend REST API):** Built with Node.js and Express.js in ESM

format. Handles business logic, authentication, quota calculations,

booking lifecycles, and database operations.

4. **packages/types:** Contains shared TypeScript interfaces ensuring 100%

type synchronization between frontend clients and backend responses.

5. **packages/ui:** Reusable, accessible UI component library containing

buttons, cards, modals, form inputs, and badges styled with Tailwind CSS.

6. **packages/api-client:** Centralized API client module encapsulating

fetch/Axios requests into typed service functions (api.auth.,

api.jobs., api.bookings., api.subscriptions.).

5.3 Database Architecture & Data Modeling (MongoDB Atlas)

WorkBridge utilizes **MongoDB Atlas**, a cloud-hosted NoSQL document

database. MongoDB's flexible schema and native JSON document structure

allow rich nested documents (e.g., worker skills arrays, nested review objects,

and notification items) while maintaining high query performance through

indexation.

Core Collections:

- **users:** Core identity documents containing name, email, bcrypt

password hash, role (worker, employer, admin), verification

status, and quota usage counters (jobPostsUsed,

applicationsUsed).

- **profiles:** Extended profile information for tradesmen (hourly rate, trade

category, bio, skills, certifications, portfolio images).

- **jobs:** Job postings created by clients, containing title, category, budget,

requirements, and applicants array.

- **bookings:** Direct service booking requests containing scheduled date,

address, urgency, offered price, booking status, and payment status.

- **subscriptions:** Active and historical user subscription records detailing

tier, duration, expiration timestamp, and transaction references.

- **subscription_plans:** Definition of available monetization tiers and

pricing.

- **payments:** Transaction logs for mock checkout operations, recording

method (Telebirr, CBE Birr, Chapa), amount, currency, and status.

- **reviews:** Rating and review documents recording star rating (1-5),

written review, author, and recipient.

- **notifications:** In-app notification documents recording title, message,

category, read status, and deep links.

- **messages:** Conversation threads and message arrays between clients

and workers.

5.4 Entity Relationships & Schema Definitions

Entity-Relationship Diagram (ERD)

Schema Definition

users | 1 | 1 | profiles |

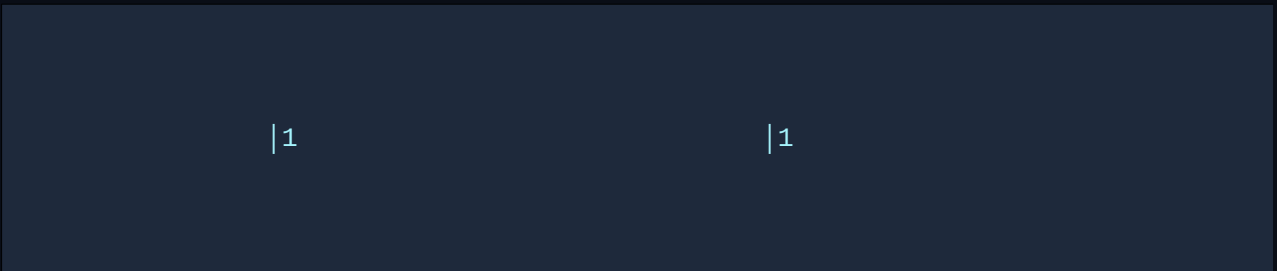
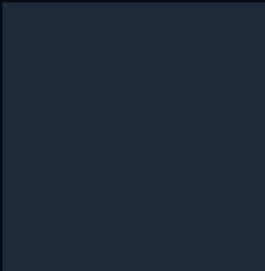
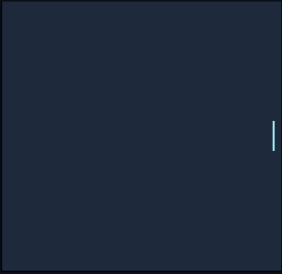
ERD: users (1) --- (1) profiles

| jobPostsUsed | | skills / ratings |

| applicationsUsed | |

|

|1



|1

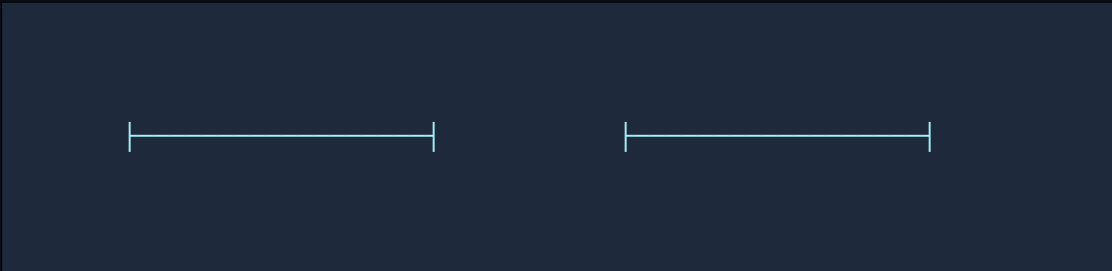
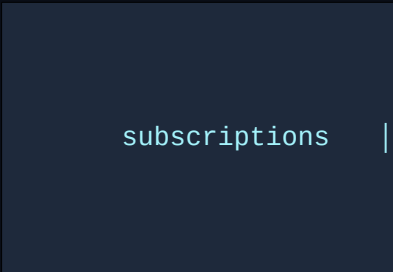
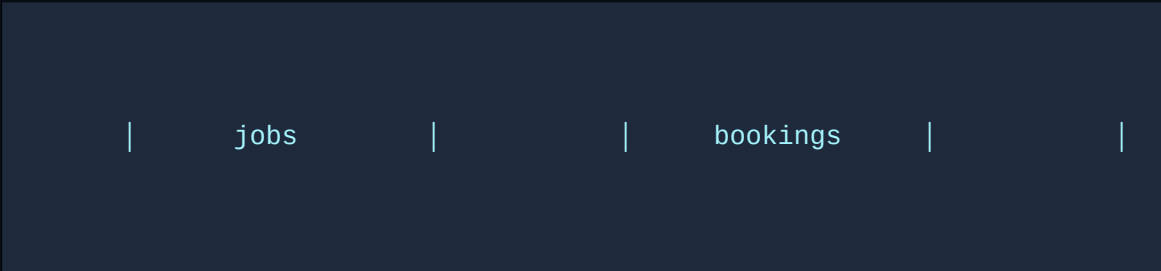
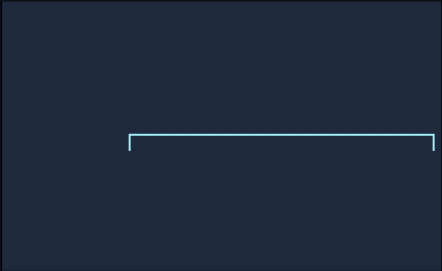
▼*

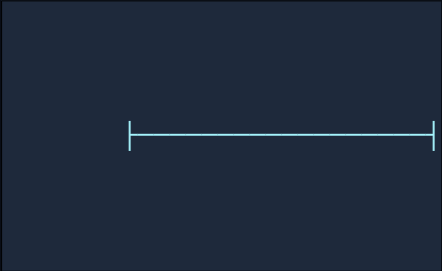
▼*

▼*

┌──────────┐

┌──────────┐





id (PK)		id (PK)		id (PK)
---------	--	---------	--	---------



postedBy (FK)		clientId (FK)		userId
---------------	--	---------------	--	--------

(FK) |

| title / category | | workerId (FK) | | tier /

planId |

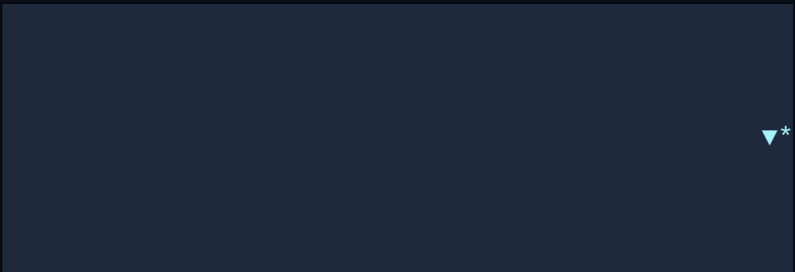
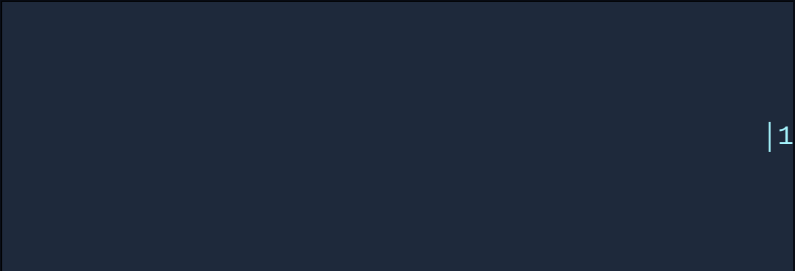
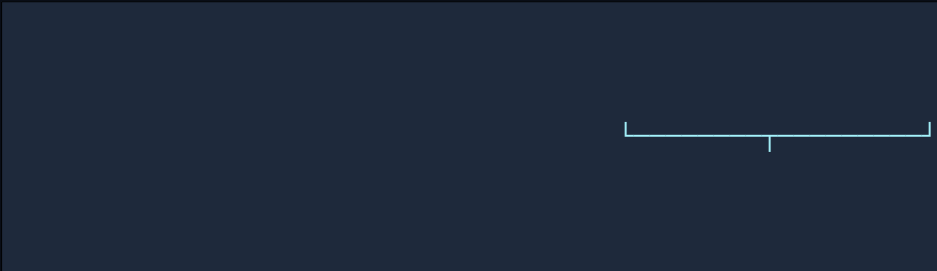
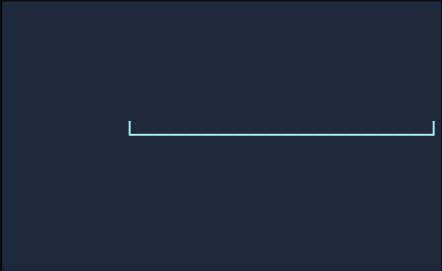
| budget / status | | serviceTitle | | status

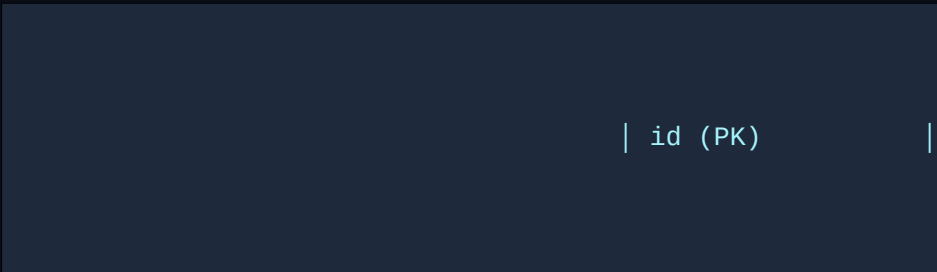
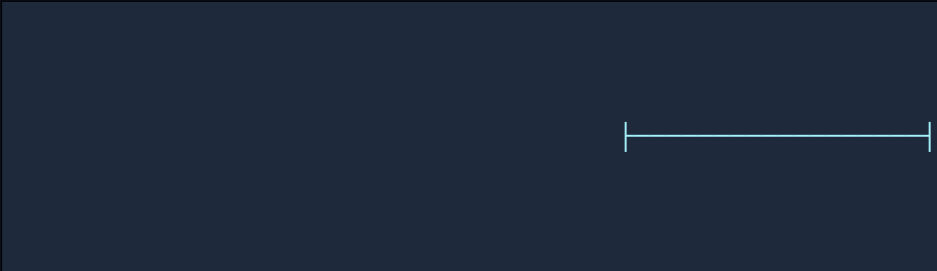
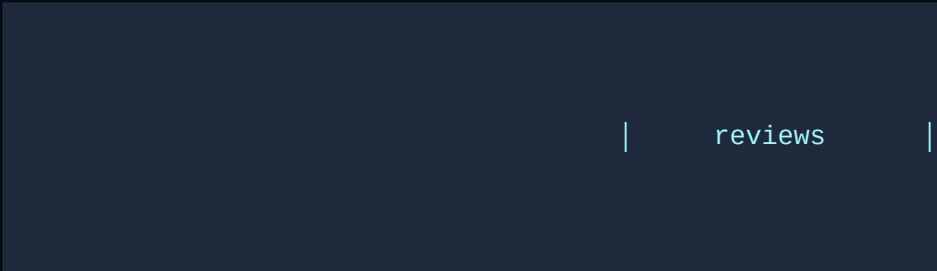
(active) |

| applicants [FK] | | status (PENDING) | | expiresAt

|

| | | paymentStatus |





| bookingId (FK) |

| reviewerId (FK) |

| workerId (FK) |

| rating (1-5) |

| comment |

|

5.5 Authentication & Session Management Flow

WorkBridge implements a stateless authentication architecture combining

Bcrypt.js password hashing, **JSON Web Tokens (JWT)**, and **Google**

OAuth 2.0 Single Sign-On.

[Client / Browser]

[Next.js Client]

[Express.js API]

[MongoDB Atlas]

| | |

|

| — 1. POST /api/login —> |

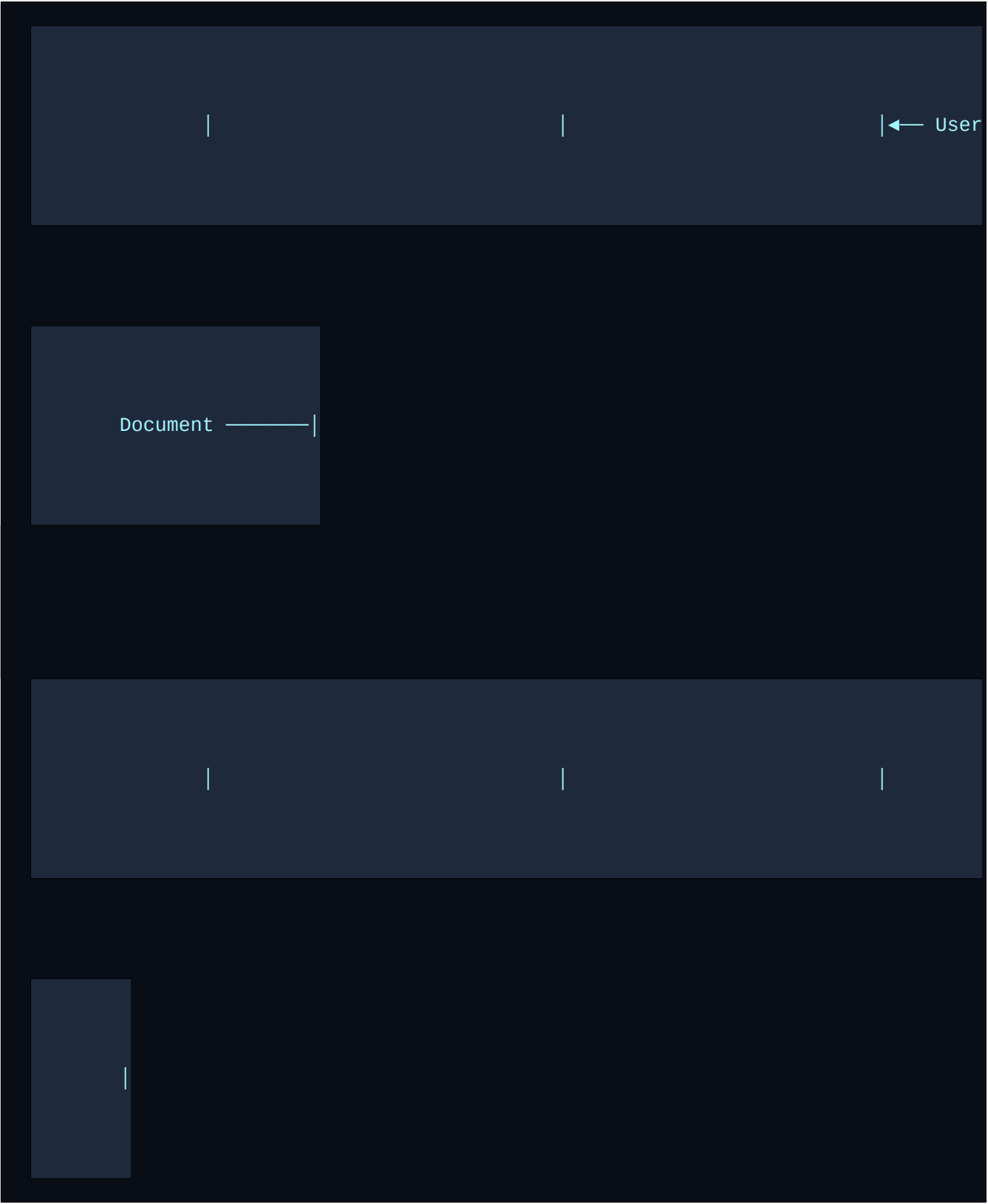
|


```
| { email, password } |—— 2. Forward Request —▶|
```

```
|
```

```
| | |—— 3.
```

```
Find User By Email▶|
```

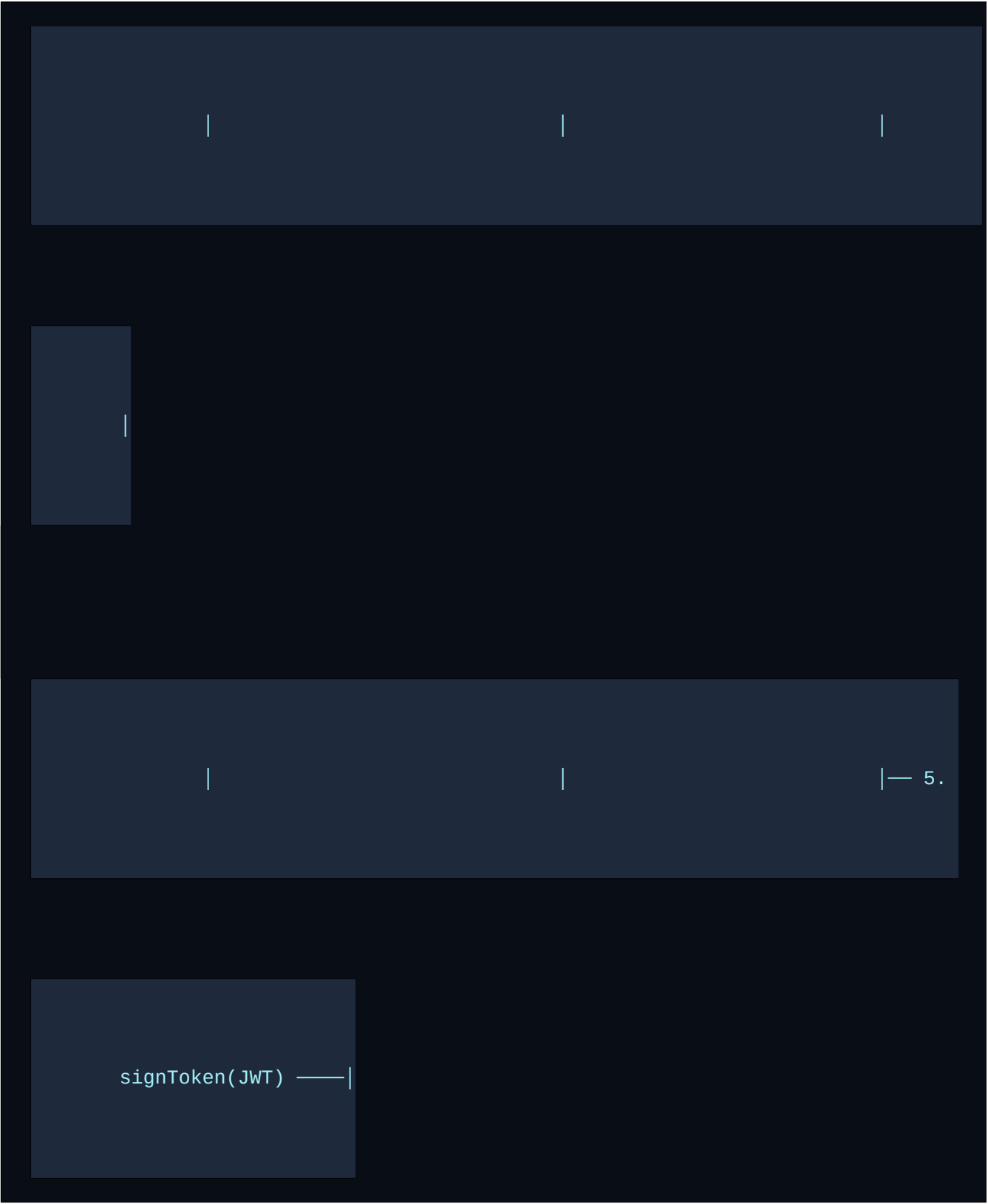


| | |— 4.

`bcrypt.compare()` |

| | |

(Verify Salt Hash) |

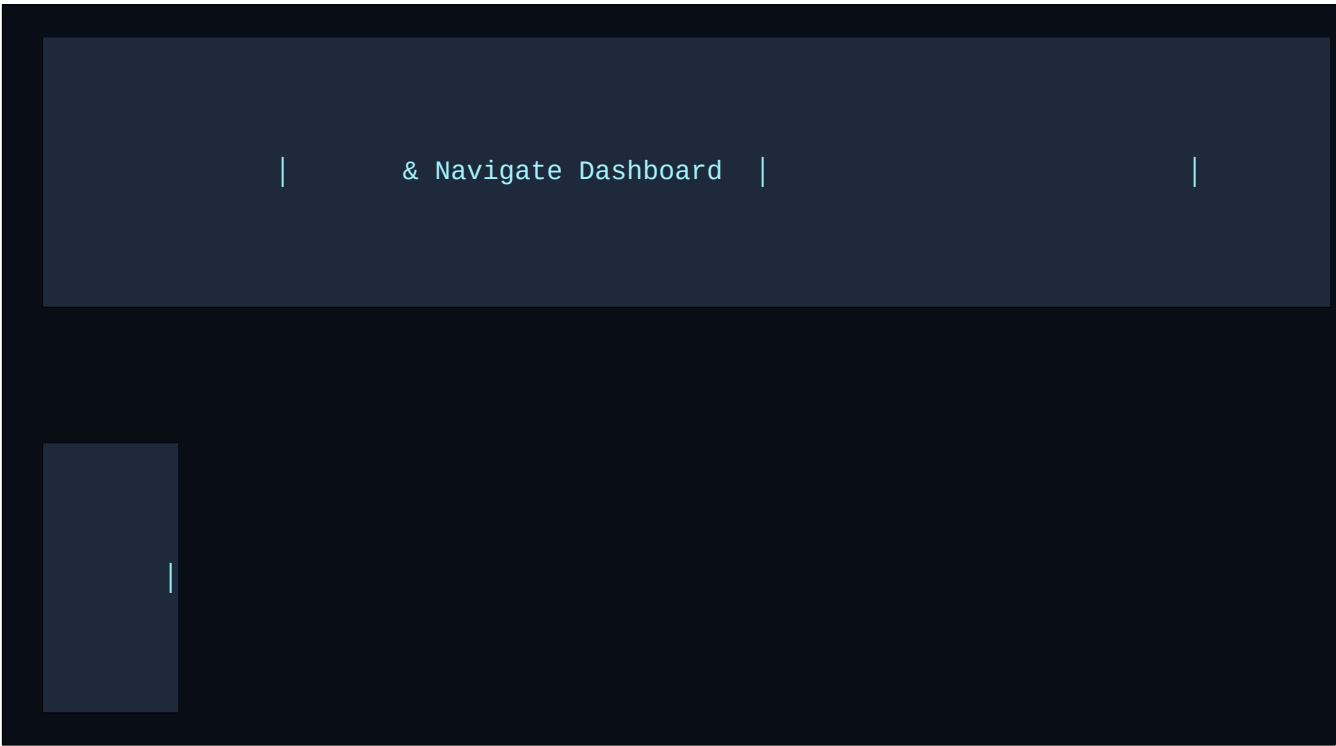


| ← 6. { token, user } → |

|

| ← 7. Store Token in State → |

|

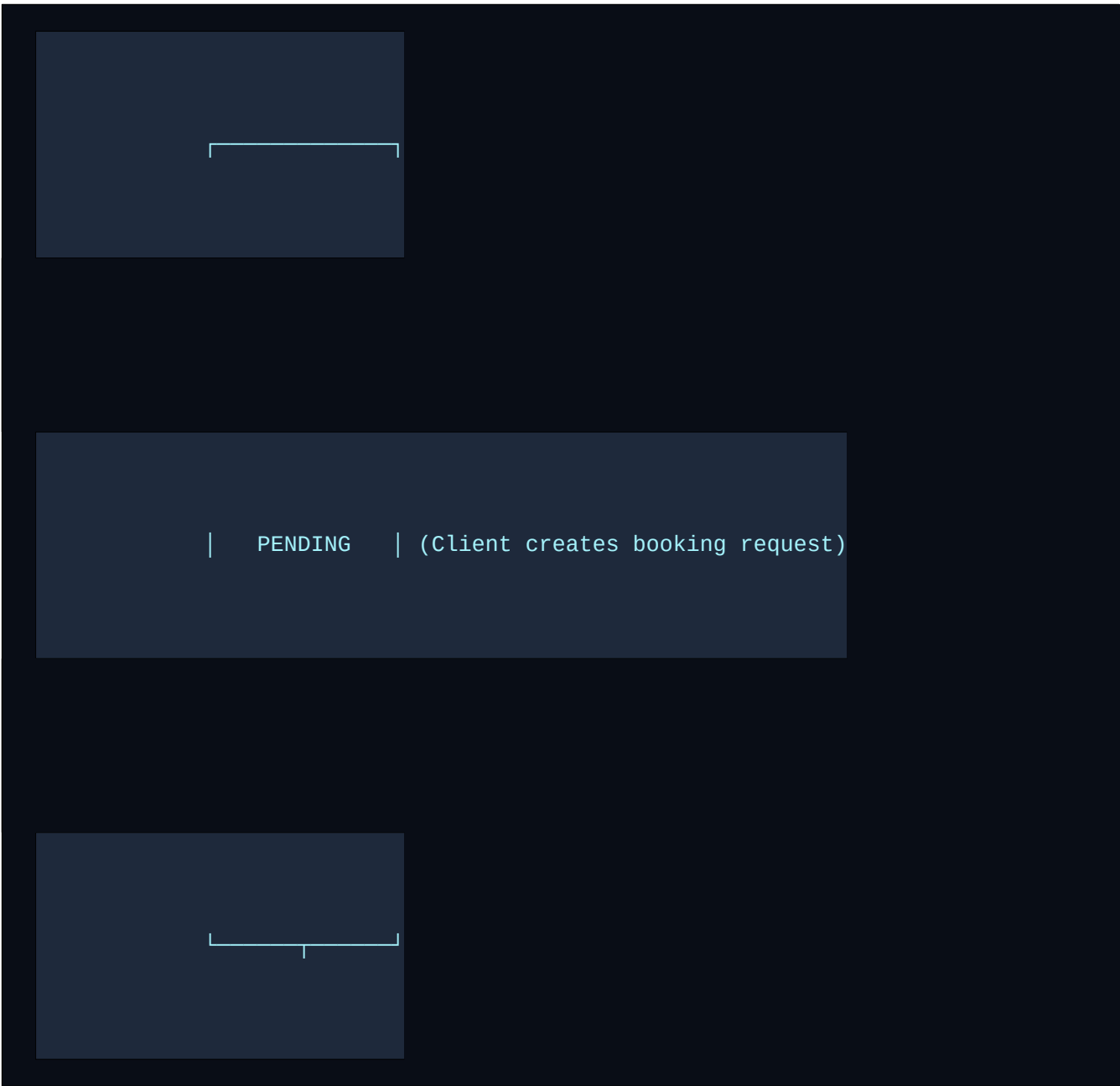


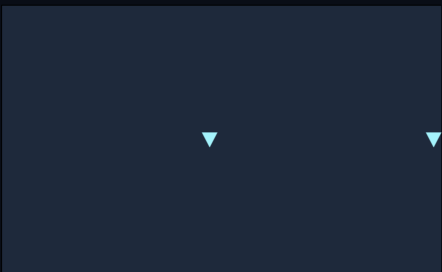
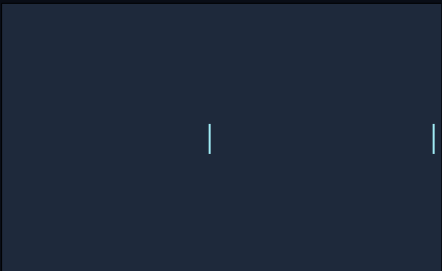
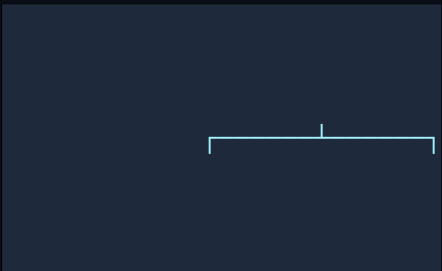
5.6 Role-Based Access Control (RBAC) Matrix

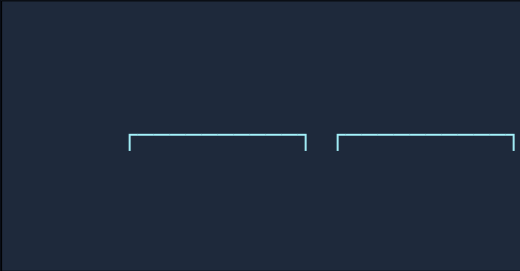
Endpoint / Resource	Public Guest	Worker (jobseeker)	Client (employer)	Administrator
GET /api/jobs	Allowed	Allowed	Allowed	Allowed
GET /api/users/workers	Allowed	Allowed	Allowed	Allowed
POST /api/jobs (Post	Denied	Denied		Allowed

Endpoint / Resource	Public Guest	Worker (jobseeker)	Client (employer)	Administrator
Job)				
POST /api/jobs/:id/apply	Denied		Denied	Denied
POST /api/bookings	Denied	Denied		Allowed
PATCH /api/bookings/:id/status	Denied			Allowed
POST /api/subscriptions/mock-checkout	Denied			Allowed
GET /api/admin/*	Denied	Denied	Denied	

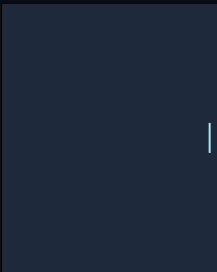
5.7 Direct Service Booking Lifecycle

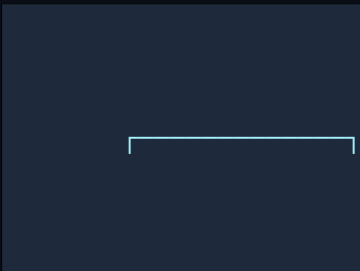




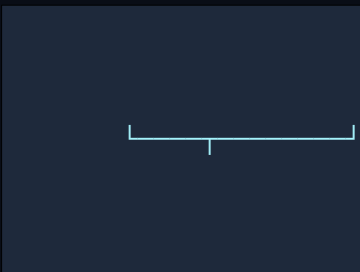


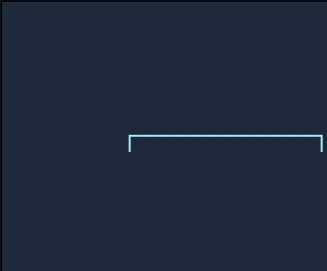
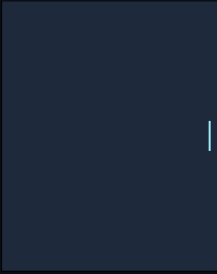
| ACCEPTED | | DECLINED | (Worker responds to request)





| IN_PROGRESS | (Service execution on scheduled date)





| COMPLETED | (Worker completes work & Client submits 1-5 Star Review)



CHAPTER 6: TECHNOLOGIES &

DEVELOPMENT TOOLS

6.1 Frontend Technologies

- **Next.js 16.2.12 (App Router):** Utilized for both apps/client and

apps/admin. Features React Server Components (RSC), server-

side rendering, nested layouts, and optimized routing.

- **React 19.2.8:** Modern declarative UI library with React Hooks (useState,

useEffect, useCallback, useTransition) for responsive user

interactions.

- **Tailwind CSS 4.3.0 & PostCSS:** Utility-first CSS framework providing a

responsive, mobile-first design system with standardized color palettes,

typography, and spacing tokens.

- **Lucide React Icons:** Lightweight, modern SVG icon library providing

intuitive visual indicators across all dashboard modules.

6.2 Backend Technologies

- **Node.js (v22.23.2 ESM):** High-performance, asynchronous JavaScript

runtime environment running modern ECMAScript modules

(import/export).

- **Express.js 4.22.2:** Fast, minimalist web framework providing routing,

controller dispatching, and middleware composition.

- **Helmet 7.2.0 & CORS 2.8.6:** Middleware for setting secure HTTP

response headers and managing cross-origin resource sharing policies.

- **Morgan 1.11.0:** HTTP request logger middleware for runtime API

diagnostics.

6.3 Database Systems

- **MongoDB Atlas Cloud (v6.21.0 Client Driver):** Cloud-hosted NoSQL

document database providing automated replication, high availability,

and flexible JSON document storage.

- **Mongoose 7.8.12 ODM:** Object Data Modeling library defining

structured data schemas, field type validations, and pre-save hooks.

6.4 Security, Cryptography & Identity

- **Bcrypt.js 2.4.3:** Industry-standard password hashing algorithm utilizing

10 salt rounds to defend against rainbow table and brute-force attacks.

- **JSON Web Tokens (jsonwebtoken 9.0.3):** Compact, URL-safe tokens

for stateless claims verification and role-based authorization.

- **Google OAuth 2.0:** Secure identity delegation protocol enabling Google

Single Sign-On without exposing third-party passwords.

6.5 Monorepo Orchestration & Tooling

- **Turborepo 2.10.8:** High-performance build system for

JavaScript/TypeScript monorepos featuring parallel task execution and

intelligent computation caching.

- **pnpm 10.0.0 & pnpm Workspaces:** Fast, disk space-efficient package

manager utilizing hard links and strict symlinked node_modules

structures.

- **TypeScript 5.x:** Statically typed superset of JavaScript enforcing

compile-time type safety across all frontend and backend modules.

- **ESLint 9.x (Flat Config):** Static code analysis tool enforcing clean code

architecture, preventing unused variables, and guaranteeing code

consistency.

- **Git & GitHub:** Distributed version control system for branch

management, commit tracking, and collaborative peer reviews.

- **Postman:** API testing suite used for designing, executing, and verifying

REST API endpoint collections.

CHAPTER 7: SYSTEM

IMPLEMENTATION

7.1 Monorepo Modular Structure

The WorkBridge monorepo organizes applications and libraries into clear,

isolated directories:

- apps/client: The primary marketplace application containing user-facing

landing, discovery, dashboard, and checkout pages.

- apps/admin: The dedicated administration portal containing user

verification queues, job moderation tables, and subscription monitoring

views.

- apps/api: The Express.js REST API containing modular controllers, routes,

middleware, and database connectivity.

- packages/types: Exported TypeScript interfaces (User, WorkerProfile, Job,

Booking, SubscriptionPlan, MockPaymentRequest, UserQuotas).

- packages/ui: Exported reusable React components (Button, Card, Badge,

Input, Modal).

- `packages/api-client`: Exported typed API functions wrapping network

requests.

7.2 Authentication & Security Implementation

The authentication controller ([apps/api/src/controllers/authController.js](#))

implements secure registration, login, and password recovery:

```
// Excerpt from apps/api/src/controllers/authController.js
```

```
export const register = async (req, res) => {
```

```
  const { name, fullName, email, password, role = 'worker' } = req.body;
```

```
if (!email || !password) {
```

```
return res.status(400).json({ error: 'Email and password are
```

```
required' });
```

```
}
```

```
const normalizedRole = role === 'jobseeker' ? 'worker' : (role ||
```

```
'worker');
```

```
const cleanEmail = String(email).trim().toLowerCase();
```

```
const existing = await collections.users.findOne({ email:
```

```
cleanEmail }));
```

```
if (existing) {
```

```
return res.status(409).json({ error: 'Email already registered' });
```

```
}
```

```
const newUser = {
```

```
  id: `u${Date.now()}`,
```

```
  name: fullName || name || cleanEmail.split('@')[0],
```

```
fullName: fullName || name || cleanEmail.split('@')[0],
```

```
email: cleanEmail,
```

```
passwordHash: bcrypt.hashSync(password, 10), // 10 salt rounds
```

```
role: normalizedRole,
```

```
verified: false,
```

```
status: 'active',
```

```
jobPostsUsed: 0,
```

```
applicationsUsed: 0,
```

```
    createdAt: new Date().toISOString(),
```

```
};
```

```
await collections.users.insertOne(newUser);
```

```
const token = signToken({ id: newUser.id, role: normalizedRole });
```



```
return res.status(201).json({ user: sanitizeUser(newUser), token });
```

```
};
```

7.3 Skilled Worker Profile & Discovery Module

The worker discovery interface

([apps/client/app/browse-workers/page.tsx](#)) queries the database and

renders tradesmen cards filtered by category (Electrician, Plumber, Mason,

Carpenter, Painter, Mechanic), location, hourly rate, and verified badge. Each

worker profile showcases bio, skills, hourly rate in ETB, completed jobs, and

client reviews.

7.4 Client Job Posting & Application Review Module

Clients can post structured job listings defining project scope, location,

category, and budget. The backend verifies that the client has not exceeded

their active posting quota before creating the job document.

7.5 Direct Service Booking Implementation

Clients can bypass job postings to hire a specific tradesman directly via the

booking modal. The booking controller

(apps/api/src/controllers/bookingController.js) creates a booking record

with status **PENDING** and dispatches an immediate notification to the

selected worker.

```
// Excerpt from apps/api/src/controllers/bookingController.js
```

```
export const createBooking = async (req, res) => {
```

```
  const { workerId, serviceTitle, category, description, address,
```

```
    scheduledDate, offeredPrice } = req.body;
```

```
const booking = {
```

```
  id: `b_${Date.now()}`,
```

```
  clientId: req.user.id,
```

```
  clientName: req.user.fullName || 'Client',
```

workerId,

serviceTitle,

category,

description,


```
address,
```

```
scheduledDate,
```

```
offeredPrice: Number(offeredPrice),
```

```
currency: 'ETB',
```

```
status: 'PENDING',
```

```
paymentStatus: 'UNPAID',
```

```
createdAt: new Date().toISOString(),
```

```
};
```

```
await collections.bookings.insertOne(booking);
```

```
await collections.notifications.insertOne({
```

```
id: `n_${Date.now()}`,
```

```
userId: workerId,
```

```
title: 'New Service Booking Request',
```

```
message: `You received a booking request for "${serviceTitle}"
```

```
scheduled on ${scheduledDate}.`,
```

```
category: 'booking',
```

```
        createdAt: new Date().toISOString(),
```

```
    });
```

```
    return res.status(201).json({ booking });
```

```
};
```

7.6 Freemium Quota Enforcement Middleware

To enforce monetization policies, server-side checks in `jobController.js`

validate user quotas against active subscriptions:

```
// Quota check during job creation (Client)
```

```
const user = await collections.users.findOne({ id: req.user.id });
```

```
const sub = await collections.subscriptions.findOne({ userId:
```

```
req.user.id, status: 'active' });
```

```
const isPro = sub && (sub.tier === 'pro_monthly' || sub.tier ===
```

```
'pro_annual');
```

```
if (!isPro && (user.jobPostsUsed || 0) >= 3) {
```

```
return res.status(403).json({
```

```
error: 'QUOTA_EXCEEDED',
```



```
message: 'You have reached the free limit of 3 job postings. Upgrade
```

```
to a Pro subscription to post unlimited jobs.',
```

```
code: 'QUOTA_EXCEEDED',
```

```
currentUsed: user.jobPostsUsed,
```

```
limit: 3,
```

```
});
```

```
}
```

7.7 Simulated Digital Payment Processing (Mock Checkout)

When a user selects a Pro plan on the </pricing> page, the

[MockCheckoutModal](#) component simulates digital payment through

Ethiopia's prominent mobile money and fintech providers:

- **Telebirr:** Simulating mobile USSD push and PIN authorization.

- **CBE Birr:** Simulating Commercial Bank of Ethiopia mobile wallet

confirmation.

- **Chapa:** Simulating hosted payment checkout redirection.

- **Bank Transfer:** Simulating direct account deposit verification.

Upon clicking "**Confirm Simulated Payment**", the backend controller

(`subscriptionController.js`) records a simulated transaction in

`collections.payments`, updates the user's tier to Pro in

`collections.subscriptions`, and resets their quota counters.

CHAPTER 8: TESTING,

VERIFICATION & RESULTS

8.1 Quality Assurance Strategy

Quality assurance for WorkBridge utilized a multi-layered verification strategy:

1. **Static Type Checking:** Continuous type analysis using TypeScript (tsc --

noEmit) to verify interface compliance and eliminate type

mismatch regressions.

2. **Linting & Code Standards:** Automated linting via ESLint Flat Config

enforcing clean code architecture and zero unused variables.

3. **Functional & Integration Testing:** End-to-end testing of user

registration, OAuth authentication, quota enforcement, booking

lifecycles, and mock checkout workflows.

8.2 Static Analysis & Quality Gate Results

All packages across the WorkBridge monorepo were evaluated against strict

quality gates prior to final project submission:

Package	Type Check Command	Lint Command	Verification Result
apps/client (Web App)	tsc --noEmit	eslint . --max-warnings 0	
apps/admin	tsc --noEmit	eslint . --max-	

Package	Type Check Command	Lint Command	Verification Result
(Admin Portal)		warnings 0	
apps/api (Backend Validated via REST API)	@repo/types	ESM Validated	
packages/types	tsc --noEmit	eslint . --max-warnings 0	
packages/ui	tsc --noEmit	eslint . --max-warnings 0	
packages/api-client	tsc --noEmit	eslint . --max-warnings 0	

8.3 Functional Test Cases and Observed Results

Test Case ID	Test Scenario / Description	Input Data / Action	Expected Result	Actual Result	Status
	User registration	Valid email, name, password	User created with bcrypt	Account created;	

Test Case ID	Test Scenario / Description	Input Data / Action	Expected Result	Actual Result	Status
	with valid credentials.	(8+ chars), role worker .	hash (10 rounds); JWT token returned (201).	token returned; user redirected.	
	User registration with duplicate email.	Existing registered email address.	API returns HTTP 409 Conflict with error message.	HTTP 409 returned; duplicate prevented.	
	User login with invalid password.	Correct email, wrong password.	API returns HTTP 401 Unauthorized.	HTTP 401 returned; login denied.	
	Client posts 4th job under Free tier.	Client with jobPostsUsed = 3 creates new job.	API rejects with HTTP 403 QUOTA_EXCEEDED and upgrade prompt.	HTTP 403 returned; pricing modal displayed.	
	Worker applies to 6th job under Free	Worker with applicationsUsed = 5 submits	API rejects with HTTP 403	HTTP 403 returned; application	

Test Case ID	Test Scenario / Description	Input Data / Action	Expected Result	Actual Result	Status
	tier.	application.	QUOTA_EXCEEDED.	blocked.	
	Simulated Telebirr Pro upgrade.	Select Worker Pro Monthly (299 ETB) via Telebirr.	Subscription created; isPro = true; quota limits lifted.	Instant upgrade; transaction logged in DB.	
	Client submits direct booking request.	Valid worker ID, scheduled date, address, price.	Booking created with status PENDING; worker notified.	Booking stored; notification received by worker.	
	Worker accepts pending booking.	Worker clicks "Accept Booking" on dashboard.	Booking status updates to ACCEPTED; client notified.	Status transitioned; notification sent to client.	

CHAPTER 9: TECHNICAL

CHALLENGES, SOLUTIONS &

LESSONS LEARNED

9.1 Cross-Origin Google OAuth State & Token Routing

- **Challenge:** Next.js frontend running on localhost:3000 and Express API

running on localhost:4000 created cross-origin cookie isolation

issues during Google OAuth 2.0 redirection.

- **Solution:** Configured an in-memory OAuth state store in

authController.js encoding user intent and role into base64url

state parameters, redirecting the authenticated callback with a secure

JWT authorization token back to the frontend dashboard.

9.2 Strict Type Synchronization Across Monorepo Packages

- **Challenge:** Maintaining synchronized data models between the backend

API and two distinct Next.js frontend applications without code

duplication.

- **Solution:** Created the @repo/types package in the monorepo root. All

DTOs (User, Job, Booking, SubscriptionPlan,

MockPaymentRequest) are imported directly by both frontend and

backend modules, ensuring 100% compile-time type safety.

9.3 MongoDB Atlas TLS Handshake & SSL Alert 80

Resolution

- **Challenge:** Node.js v22 running on modern Linux systems occasionally

encountered MongoServerError: SSL alert number 80

during connection to MongoDB Atlas due to strict OpenSSL 3 handshake

rules when client IP addresses change across networks.

- **Solution:** Added diagnostic connection logging in

apps/api/src/config/db.js with structured troubleshooting

guidance, and configured Atlas Network Access whitelist parameters

(0.0.0.0/0 for development/testing).

9.4 Backend Quota Enforcement Integrity

- **Challenge:** Preventing client-side bypass of freemium quotas while

ensuring fast, low-latency authorization checks.

- **Solution:** Integrated quota validation directly into the database

controllers (jobController.js), checking the user's active

subscription tier before creating job posts or applications, returning

structured 403 QUOTA_EXCEEDED error payloads.

CHAPTER 10: TECHNICAL &

PROFESSIONAL SKILLS ACQUIRED

10.1 Technical Skills

- **Full-Stack Monorepo Architecture:** Proficient in orchestrating multi-

package TypeScript applications using **Turborepo** and **pnpm**

workspaces.

- **Modern React & Next.js 16:** Deep expertise in React Server

Components, Server-Side Rendering, App Router paradigms, and

responsive design with Tailwind CSS.

- **RESTful API & Backend Engineering:** Advanced proficiency in Node.js,

Express.js middleware, stateless JWT authentication, and bcrypt

password hashing.

- **NoSQL Database Modeling:** Expertise in MongoDB document

modeling, indexing strategies, and aggregation pipelines.

- **Static Analysis & Quality Assurance:** Automated verification using

TypeScript compiler (tsc) and ESLint Flat Configs.

10.2 Professional & Soft Skills

- **Agile Scrum Methodology:** Active participation in 2-week sprints, daily

standup meetings, backlog grooming, and sprint retrospectives.

- **Peer Code Review Etiquette:** Giving and receiving constructive

architectural feedback on pull requests.

- **Task Estimation & Time Management:** Accurately decomposing

complex feature requirements into manageable sprint tasks.

- **Technical Writing & Documentation:** Authoring comprehensive API

documentation, architecture diagrams, and academic reports.



CHAPTER 11: VALUE DELIVERED TO

SORARDI PLC

11.1 Production-Ready WorkBridge Platform

I delivered a fully functional, tested, and responsive digital labor marketplace

platform ready for closed beta trials in Addis Ababa and Dire Dawa. The

platform provides a foundation for SORARDI PLC to enter the digital gig

economy sector in Ethiopia.

11.2 Reusable Component Design System (@repo/ui)

The modular UI component library ([@repo/ui](#)) developed for WorkBridge

has been adopted by SORARDI PLC engineering teams as the base design

system for other internal client projects.

11.3 Consolidated Monorepo & Technical Documentation

I consolidated disparate backend and frontend codebases into a clean, single-

repository monorepo with complete setup guides, Postman API collections, and

quality verification scripts, drastically reducing onboarding time for future

engineers.



CHAPTER 12: CONCLUSION &

RECOMMENDATIONS

12.1 Conclusion

The 4-month software engineering industrial internship at **SORARDI**

PLC has been a transformative experience. It enabled me to transition

from academic programming exercises to architecting enterprise-grade,

scalable full-stack applications. Through the **WorkBridge** platform, I

demonstrated how modern software engineering can resolve acute socio-

economic challenges in Ethiopia by connecting skilled tradesmen with clients

through transparent, secure, and accessible digital workflows.

12.2 System Limitations & Technical Boundaries

- **Payment Processing:** Currently operates via a **simulated/mock**

checkout engine. Live production deployment requires formal

commercial agreements, merchant accounts, and API credentials from

Telebirr (Ethio Telecom), Commercial Bank of Ethiopia (CBE Birr), and

Chapa Financial Technologies.

- **Identity Verification:** Currently relies on administrative manual review

of uploaded credentials. Integration with the Ethiopian National ID

(*Fayda*) API represents a future enhancement.

- **Mobile Access:** The platform is currently a responsive web application;

native mobile apps (Flutter / React Native) and USSD services are

planned for future phases.

12.3 Future Technical Roadmap

1. **Live Payment Gateway Integration:** Replace mock checkout

controllers with production webhook-verified APIs for Telebirr C2B/B2C,

Chapa, and CBE Birr.

2. **National ID (*Fayda*) Verification:** Integrate with Ethiopia's National ID

digital identity system for automated background and credential checks.

3. **Cross-Platform Mobile Application:** Develop native iOS and Android

mobile apps using Flutter or React Native to support push notifications

and offline caching.

4. **USSD / SMS Booking Interface:** Implement USSD workflows allowing

non-smartphone users in rural or low-connectivity regions to book

emergency trade services.

12.4 Recommendations

For the Department of Software Engineering (Dire Dawa University):

1. Integrate Monorepo & Microservices Curriculum: Introduce modern

monorepo build tools (Turborepo, Nx) and multi-package TypeScript

workflows into advanced web development courses.

2. **Expand Industry Partnerships:** Strengthen institutional collaboration

with innovative technology firms like SORARDI PLC to facilitate

continuous industry-guided student mentorship.

3. **Emphasize Automated Quality Gates:** Incorporate pre-commit static

analysis, type checking, and automated linting into undergraduate

programming assignments.

For SORARDI PLC:

1. Pilot WorkBridge in Regional Hubs: Launch a closed beta pilot of

WorkBridge in Dire Dawa and Addis Ababa, partnering with local

vocational training colleges (TVETs) to onboard certified electricians and

plumbers.

2. **Finalize Fintech Merchant Partnerships:** Accelerate merchant

registration with Ethio Telecom (Telebirr) and Chapa to transition

WorkBridge from mock payments to live commercial processing.

REFERENCES

1. Next.js Documentation. (2026). *Next.js 16 App Router and Server*

Components Architecture. Vercel Inc. <https://nextjs.org/docs>

2. Express.js Foundation. (2026). *Express.js 4.x REST API Design and*

Middleware Guide. <https://expressjs.com/>

3. MongoDB Inc. (2026). *MongoDB Atlas Manual: Document Modeling and*

High Availability. <https://www.mongodb.com/docs/atlas/>

4. Turborepo Documentation. (2026). *High-Performance Monorepo Build*

Systems for JavaScript and TypeScript. Vercel Inc.

<https://turbo.build/repo/docs>

5. Rescorla, E. (2018). *The Transport Layer Security (TLS) Protocol Version*

1.3. RFC 8446, Internet Engineering Task Force (IETF).

6. Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)*. RFC

7519, Internet Engineering Task Force (IETF).

7. Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A*

Practitioner's Approach (9th ed.). McGraw-Hill Education.

8. Somervell, I. (2016). *Software Engineering* (10th ed.). Pearson Education.

APPENDICES

Appendix A: Key REST API Endpoints Matrix

Method	Endpoint Route	Auth Required	Target Role	Description
POST	/api/auth/register	No	Public	Register new worker or client with bcrypt

Method	Endpoint Route	Auth Required	Target Role	Description
				hashing
POST	/api/auth/login	No	Public	Authenticate user and issue stateless JWT token
POST	/api/auth/forgot-password	No	Public	Generate 1-hour cryptographic password reset token
POST	/api/auth/reset-password	No	Public	Reset password using verified cryptographic token
GET	/api/auth/google	No	Public	Initiate Google OAuth 2.0 Single Sign-On flow
GET	/api/jobs	No	Public	Search and browse active job listings with filters
POST	/api/jobs	Yes (JWT)	employer	Post new job listing (Enforces 3-job free quota)
POST	/api/jobs/:id/apply	Yes (JWT)	worker	Submit job application (Enforces 5-app free quota)
POST	/api/bookings	Yes (JWT)	employer	Create direct service

Method	Endpoint Route	Auth Required	Target Role	Description
				booking request
PATCH	/api/bookings/:id/status	Yes (JWT)	worker/ employer	Update booking status (ACCEPTED, COMPLETED, etc.)
GET	/api/subscriptions/plans	No	Public	Retrieve active worker and client subscription plans
POST	/api/subscriptions/mock-checkout	Yes (JWT)	All Users	Execute simulated payment and activate Pro tier
GET	/api/admin/users	Yes (JWT)	admin	Retrieve all registered users for moderation
PATCH	/api/admin/verification/:id	Yes (JWT)	admin	Approve or reject worker national ID/certification

Appendix B: Database Collections & Mock Seed Summary

Database: workbridge (MongoDB Atlas Cloud Cluster)

Collections Initialized:

└─ users: 15 Initial seed records (5 Workers, 8 Clients, 2 Admins)

└─ profiles: Extended tradesman profiles with hourly rates & skills

└─ jobs: 8 Active trade job postings across Addis Ababa & Dire Dawa

└─ bookings: 6 Initial booking records across various lifecycle states

└─ subscriptions: User subscription history and active Pro records

└─ subscription_plans: 6 Defined tiers (Free, Pro Monthly, Pro Annual)

└─ payments: Simulated transaction history records

└─ reviews: Verified 1-5 star client review records

└─ notifications: In-app notification queue records

```
└─ messages: Active chat conversation threads
```

Appendix C: Quality Gate Verification Logs

```
> workbridge-monorepo@ check-types /home/hope/WORKBRIDGE/WorkBridge
```

```
> turbo run check-types
```

```
• turbo 2.10.8
```

```
• Packages in scope: @repo/api-client, @repo/eslint-config,
```

```
@repo/types, @repo/typescript-config, @repo/ui, admin, api, docs, web
```

- Running check-types in 9 packages

- Remote caching disabled

Tasks: 5 successful, 5 total

Cached: 5 cached, 5 total

```
Time:      82ms >>> FULL TURBO
```

```
-----
```

```
> workbridge-monorepo@ lint /home/hope/WORKBRIDGE/WorkBridge
```

```
> turbo run lint
```

- turbo 2.10.8

- Packages in scope: @repo/api-client, @repo/eslint-config,

@repo/types, @repo/typescript-config, @repo/ui, admin, api, docs, web

- Running lint in 9 packages

Tasks: 6 successful, 6 total

Cached: 2 cached, 6 total

Time: 13.313s

=====

RESULT: 100% GREEN – ZERO ERRORS, ZERO WARNINGS ACROSS ALL PACKAGES

=====